

notebook

April 28, 2026

1 Classification

1.1 Imports

```
[1]: import copy

from IPython.display import display
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
%matplotlib widget
plt.rcParams['figure.figsize'] = (12, 8)
plt.rcParams["figure.constrained_layout.use"] = True

from sklearn.base import clone
from sklearn.metrics import (
    accuracy_score,
    classification_report,
)
from sklearn.datasets import (
    make_moons,
    make_blobs, make_circles,
    make_classification,
    make_blobs
)
from sklearn.preprocessing import (
    StandardScaler,
    label_binarize
)
from sklearn.multiclass import OneVsRestClassifier, OneVsOneClassifier
from sklearn.model_selection import train_test_split
from sklearn.linear_model import (
    LogisticRegression,
    SGDClassifier
)
from sklearn.inspection import DecisionBoundaryDisplay
```

```

from classification.base.basemodel import BaseModel
from classification.base.basegdmmodel import BaseGDMModel
from classification.dataset import CovtypeDataset as Dataset
from classification.eda.eda import EDA
from classification.pipeline import get_pipeline
from classification.logger import get_logger

# Logistic regression
from classification.logistic_regression.softmax import SoftmaxClassifier
from classification.logistic_regression.sigmoid import SigmoidClassifier
from classification.logistic_regression.irls import IRLSClassifier

# Discriminant Analysis
from classification.lda.lda import LDA
from classification.lda.qda import QDA
from classification.lda.fisher_ratio import fisher_ratio

# Perceptron and Regularization
from classification.perceptron_logreg.perceptron import Perceptron
from classification.perceptron_logreg.class_weighted import WeightedSoftmaxClassifier
from classification.perceptron_logreg.lambda_selection import select_best_lambda

# Advanced
from classification.advanced.probit import ProbitClassifier
from classification.advanced.laplace import LaplaceApprox
from classification.advanced.kernel_logistic import KernelLogisticRegression

# Evaluation
from evaluation.evaluator import Evaluator
from evaluation.visualizer import Visualizer

# Sensitivity & Robustness
from classification.sensitivity_robustness import (
    sensitivity_split_analysis,
    plot_split_boxplots,
    noise_injection_analysis,
    feature_corruption_analysis,
    plot_loss_curves
)

# Benchmark
from classification.benchmark import benchmark_models

```

1.2 Dataset

```
[2]: d = Dataset()  
     d.split()
```

```
[3]: eda = EDA(d)
```

1.2.1 Missing values

```
[5]: display(eda.missing_values())
```

	feature \
Elevation	Elevation
Aspect	Aspect
Slope	Slope
Horizontal_Distance_To_Hydrology	Horizontal_Distance_To_Hydrology
Vertical_Distance_To_Hydrology	Vertical_Distance_To_Hydrology
Horizontal_Distance_To_Roadways	Horizontal_Distance_To_Roadways
Hillshade_9am	Hillshade_9am
Hillshade_Noon	Hillshade_Noon
Hillshade_3pm	Hillshade_3pm
Horizontal_Distance_To_Fire_Points	Horizontal_Distance_To_Fire_Points
Wilderness_Area_0	Wilderness_Area_0
Wilderness_Area_1	Wilderness_Area_1
Wilderness_Area_2	Wilderness_Area_2
Wilderness_Area_3	Wilderness_Area_3
Soil_Type_0	Soil_Type_0
Soil_Type_1	Soil_Type_1
Soil_Type_2	Soil_Type_2
Soil_Type_3	Soil_Type_3
Soil_Type_4	Soil_Type_4
Soil_Type_5	Soil_Type_5
Soil_Type_6	Soil_Type_6
Soil_Type_7	Soil_Type_7
Soil_Type_8	Soil_Type_8
Soil_Type_9	Soil_Type_9
Soil_Type_10	Soil_Type_10
Soil_Type_11	Soil_Type_11
Soil_Type_12	Soil_Type_12
Soil_Type_13	Soil_Type_13
Soil_Type_14	Soil_Type_14
Soil_Type_15	Soil_Type_15
Soil_Type_16	Soil_Type_16
Soil_Type_17	Soil_Type_17
Soil_Type_18	Soil_Type_18
Soil_Type_19	Soil_Type_19
Soil_Type_20	Soil_Type_20
Soil_Type_21	Soil_Type_21

Soil_Type_22	Soil_Type_22
Soil_Type_23	Soil_Type_23
Soil_Type_24	Soil_Type_24
Soil_Type_25	Soil_Type_25
Soil_Type_26	Soil_Type_26
Soil_Type_27	Soil_Type_27
Soil_Type_28	Soil_Type_28
Soil_Type_29	Soil_Type_29
Soil_Type_30	Soil_Type_30
Soil_Type_31	Soil_Type_31
Soil_Type_32	Soil_Type_32
Soil_Type_33	Soil_Type_33
Soil_Type_34	Soil_Type_34
Soil_Type_35	Soil_Type_35
Soil_Type_36	Soil_Type_36
Soil_Type_37	Soil_Type_37
Soil_Type_38	Soil_Type_38
Soil_Type_39	Soil_Type_39
Cover_Type	Cover_Type

	missing_count	missing_percent
Elevation	0	0.0
Aspect	0	0.0
Slope	0	0.0
Horizontal_Distance_To_Hydrology	0	0.0
Vertical_Distance_To_Hydrology	0	0.0
Horizontal_Distance_To_Roadways	0	0.0
Hillshade_9am	0	0.0
Hillshade_Noon	0	0.0
Hillshade_3pm	0	0.0
Horizontal_Distance_To_Fire_Points	0	0.0
Wilderness_Area_0	0	0.0
Wilderness_Area_1	0	0.0
Wilderness_Area_2	0	0.0
Wilderness_Area_3	0	0.0
Soil_Type_0	0	0.0
Soil_Type_1	0	0.0
Soil_Type_2	0	0.0
Soil_Type_3	0	0.0
Soil_Type_4	0	0.0
Soil_Type_5	0	0.0
Soil_Type_6	0	0.0
Soil_Type_7	0	0.0
Soil_Type_8	0	0.0
Soil_Type_9	0	0.0
Soil_Type_10	0	0.0
Soil_Type_11	0	0.0
Soil_Type_12	0	0.0

Soil_Type_13	0	0.0
Soil_Type_14	0	0.0
Soil_Type_15	0	0.0
Soil_Type_16	0	0.0
Soil_Type_17	0	0.0
Soil_Type_18	0	0.0
Soil_Type_19	0	0.0
Soil_Type_20	0	0.0
Soil_Type_21	0	0.0
Soil_Type_22	0	0.0
Soil_Type_23	0	0.0
Soil_Type_24	0	0.0
Soil_Type_25	0	0.0
Soil_Type_26	0	0.0
Soil_Type_27	0	0.0
Soil_Type_28	0	0.0
Soil_Type_29	0	0.0
Soil_Type_30	0	0.0
Soil_Type_31	0	0.0
Soil_Type_32	0	0.0
Soil_Type_33	0	0.0
Soil_Type_34	0	0.0
Soil_Type_35	0	0.0
Soil_Type_36	0	0.0
Soil_Type_37	0	0.0
Soil_Type_38	0	0.0
Soil_Type_39	0	0.0
Cover_Type	0	0.0

1.2.2 Descriptive statistics

```
[6]: result = eda.descriptive_stats_numeric_features()
# result.round(2).to_csv("covtype-desc.csv")
display(result)
```

	Elevation	Aspect	Slope \
count	581012.000000	581012.000000	581012.000000
mean	2959.365301	155.656807	14.103704
std	279.984734	111.913721	7.488242
min	1859.000000	0.000000	0.000000
25%	2809.000000	58.000000	9.000000
50%	2996.000000	127.000000	13.000000
75%	3163.000000	260.000000	18.000000
max	3858.000000	360.000000	66.000000

	Horizontal_Distance_To_Hydrology	Vertical_Distance_To_Hydrology \
count	581012.000000	581012.000000
mean	269.428217	46.418855

std	212.549356	58.295232
min	0.000000	-173.000000
25%	108.000000	7.000000
50%	218.000000	30.000000
75%	384.000000	69.000000
max	1397.000000	601.000000

	Horizontal_Distance_To_Roadways	Hillshade_9am	Hillshade_Noon \
count	581012.000000	581012.000000	581012.000000
mean	2350.146611	212.146049	223.318716
std	1559.254870	26.769889	19.768697
min	0.000000	0.000000	0.000000
25%	1106.000000	198.000000	213.000000
50%	1997.000000	218.000000	226.000000
75%	3328.000000	231.000000	237.000000
max	7117.000000	254.000000	254.000000

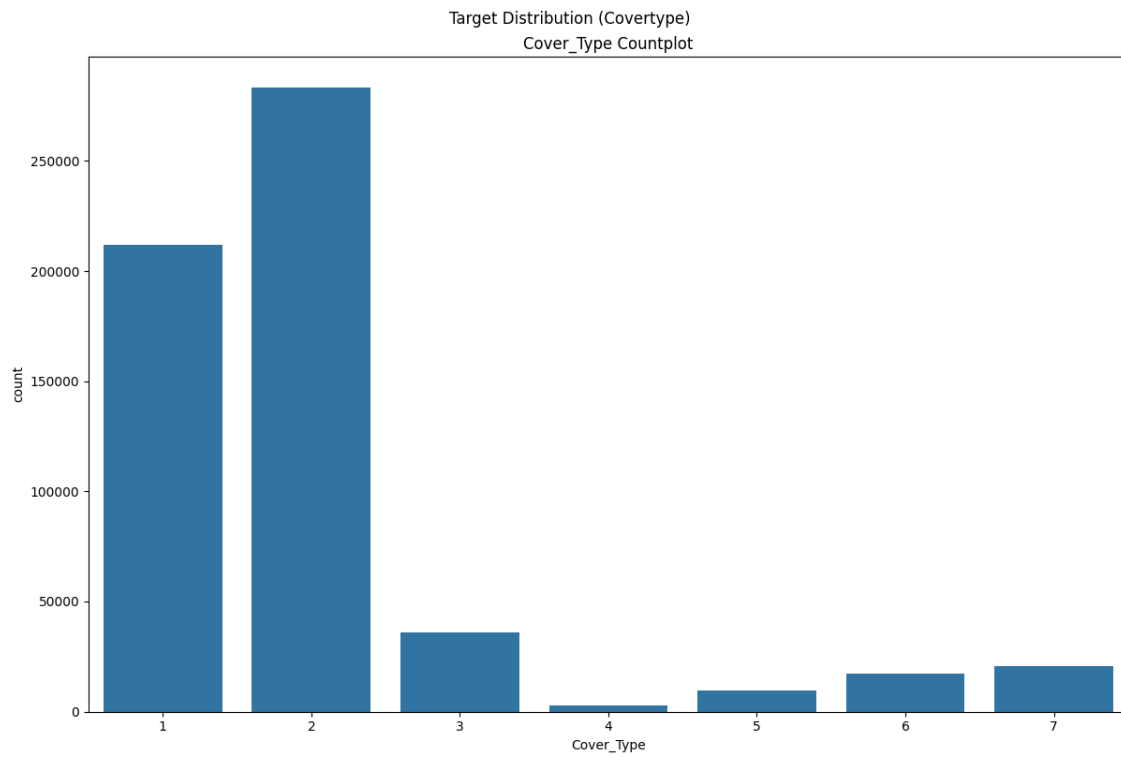
	Hillshade_3pm	Horizontal_Distance_To_Fire_Points
count	581012.000000	581012.000000
mean	142.528263	1980.291226
std	38.274529	1324.195210
min	0.000000	0.000000
25%	119.000000	1024.000000
50%	143.000000	1710.000000
75%	168.000000	2550.000000
max	254.000000	7173.000000

1.2.3 Target distribution

```
[7]: result = eda.target_distribution()
# result.to_csv("covtype-target-count.csv")
display(result)
```

Cover_Type	count	percent
1	211840	36.46
2	283301	48.76
3	35754	6.15
4	2747	0.47
5	9493	1.63
6	17367	2.99
7	20510	3.53

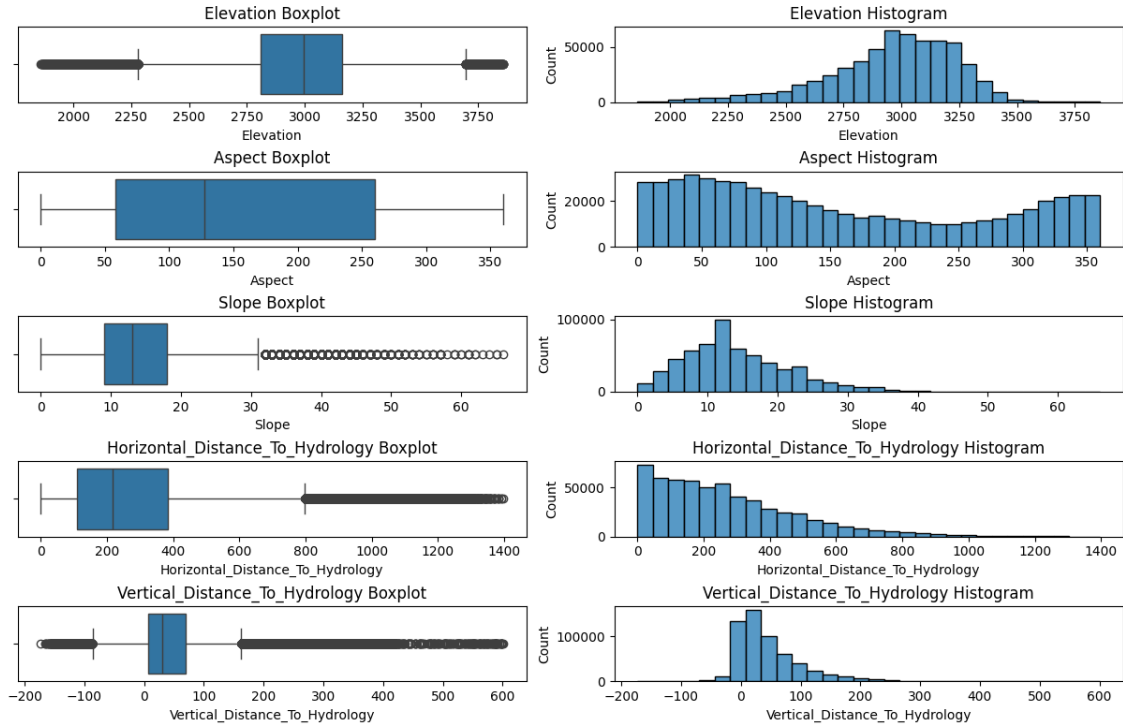
```
[8]: eda.plot_target_distribution()
plt.show()
```



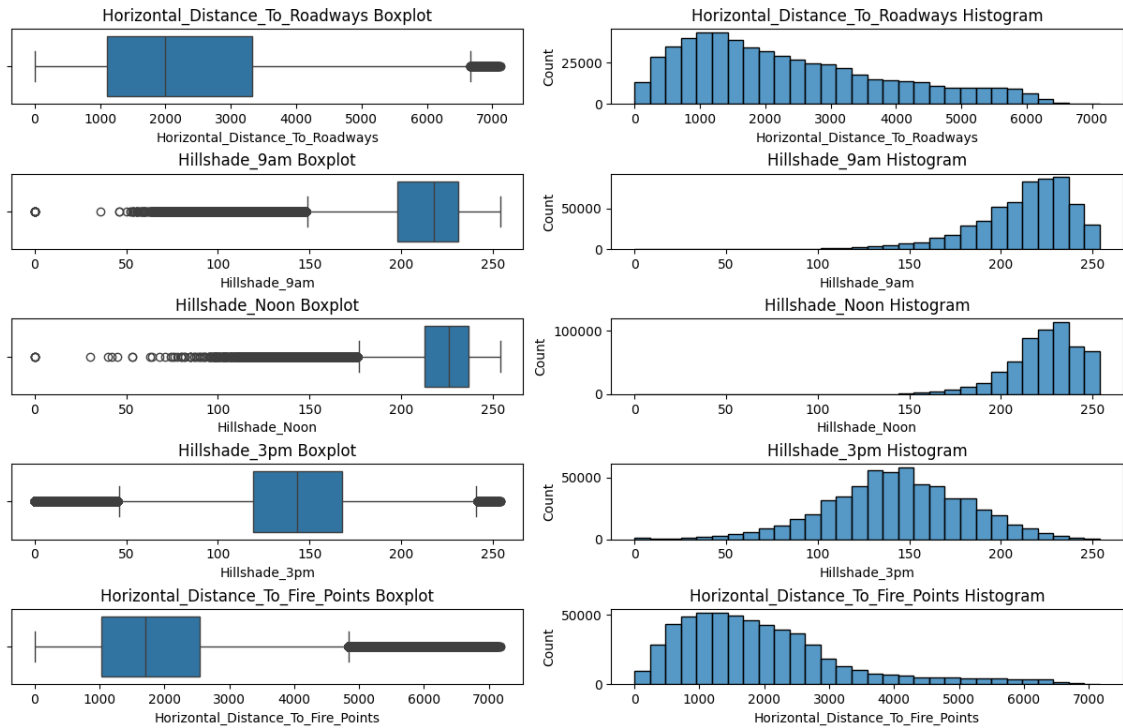
1.2.4 Numeric features

```
[9]: figs = eda.plot_numeric_features()
     for fig in figs:
         fig.show()
```

Numeric Features - Group 1

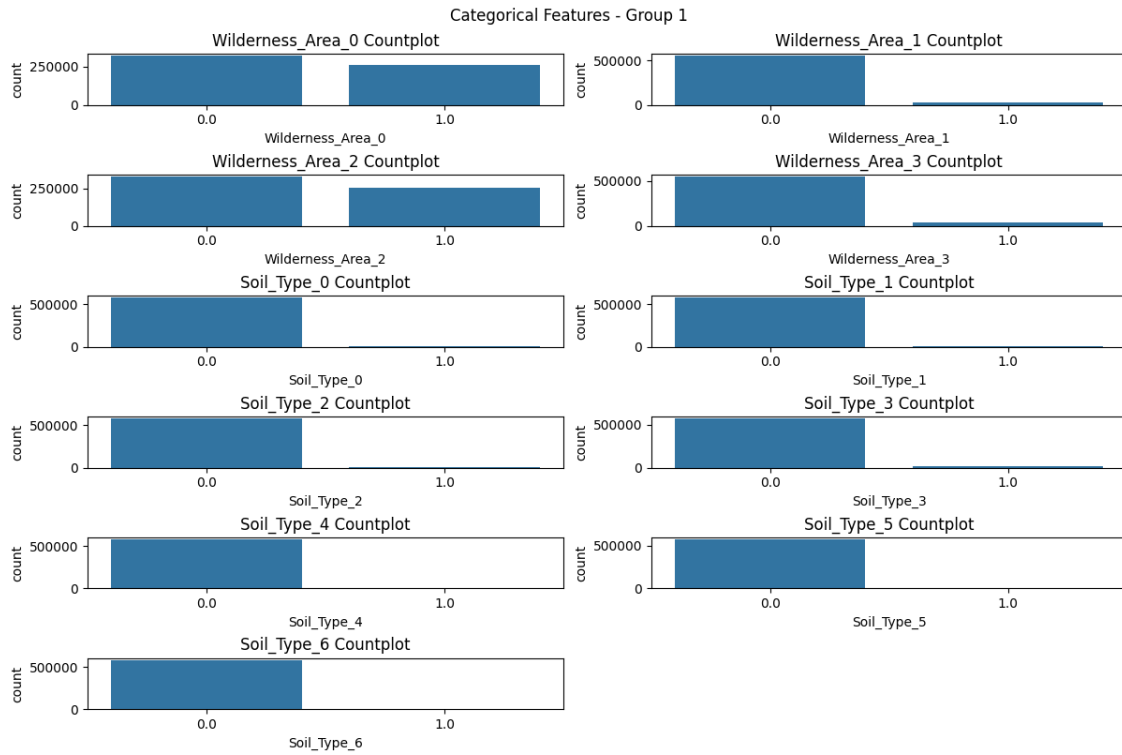


Numeric Features - Group 2

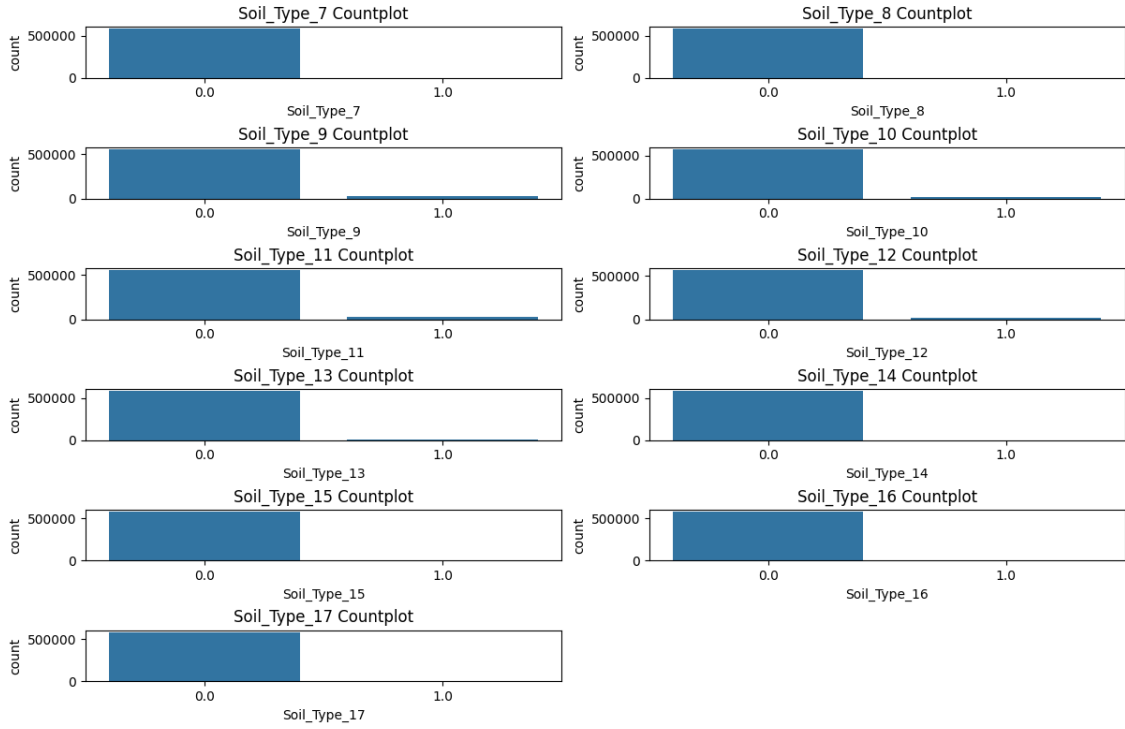


1.2.5 Categorical features

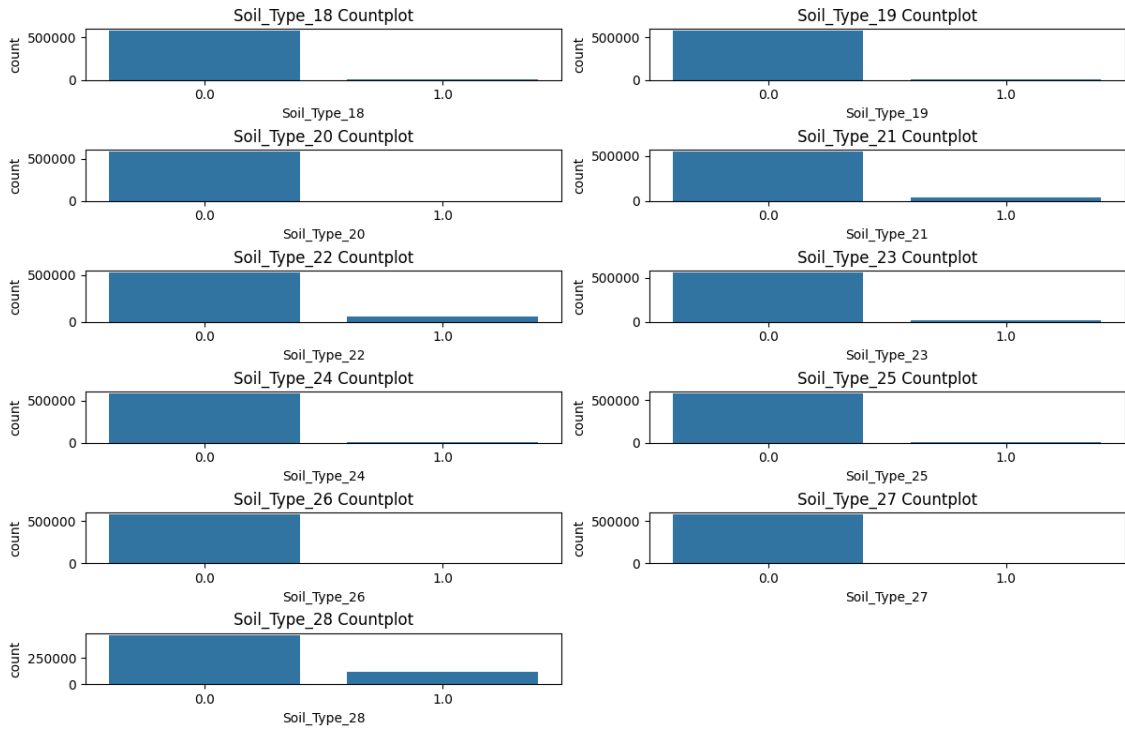
```
[4]: figs = eda.plot_categorical_features()
    for fig in figs:
        fig.show()
```

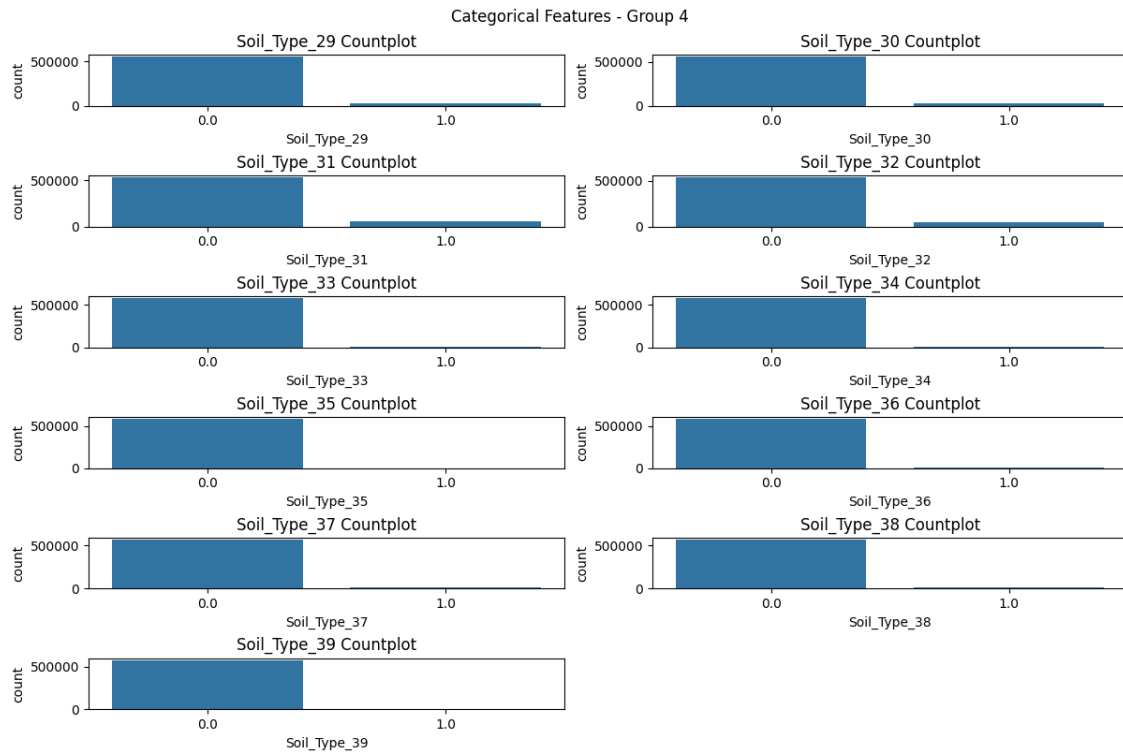


Categorical Features - Group 2



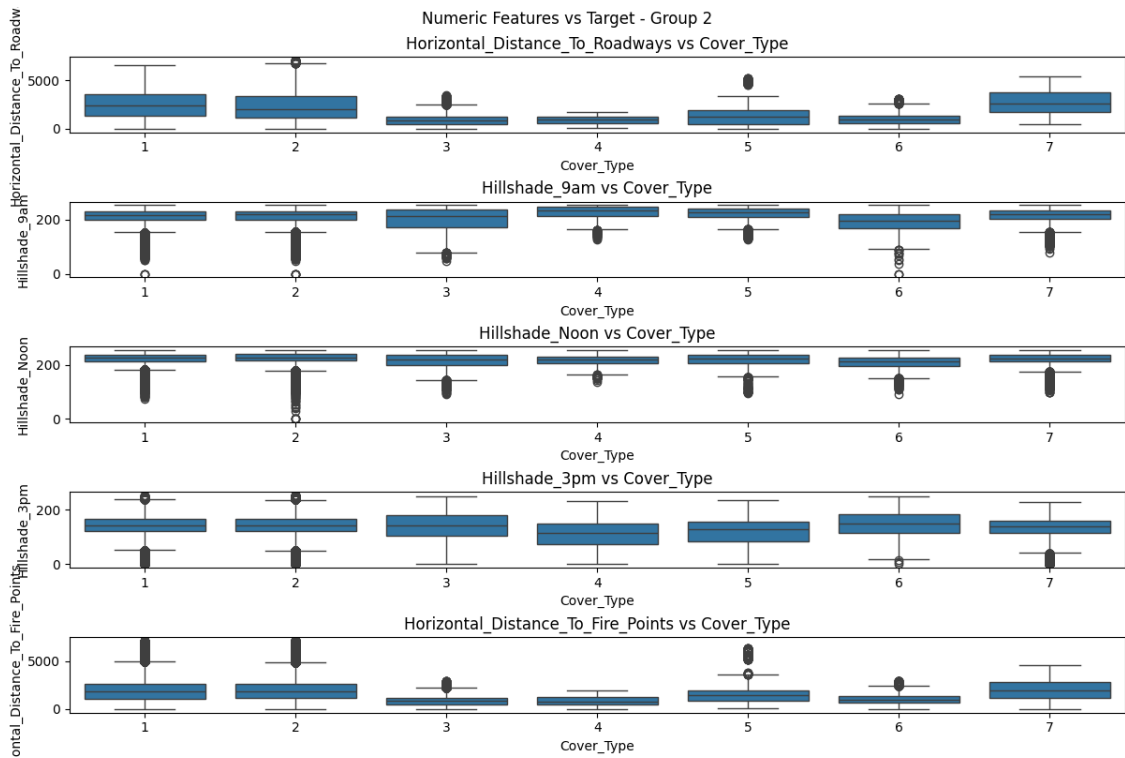
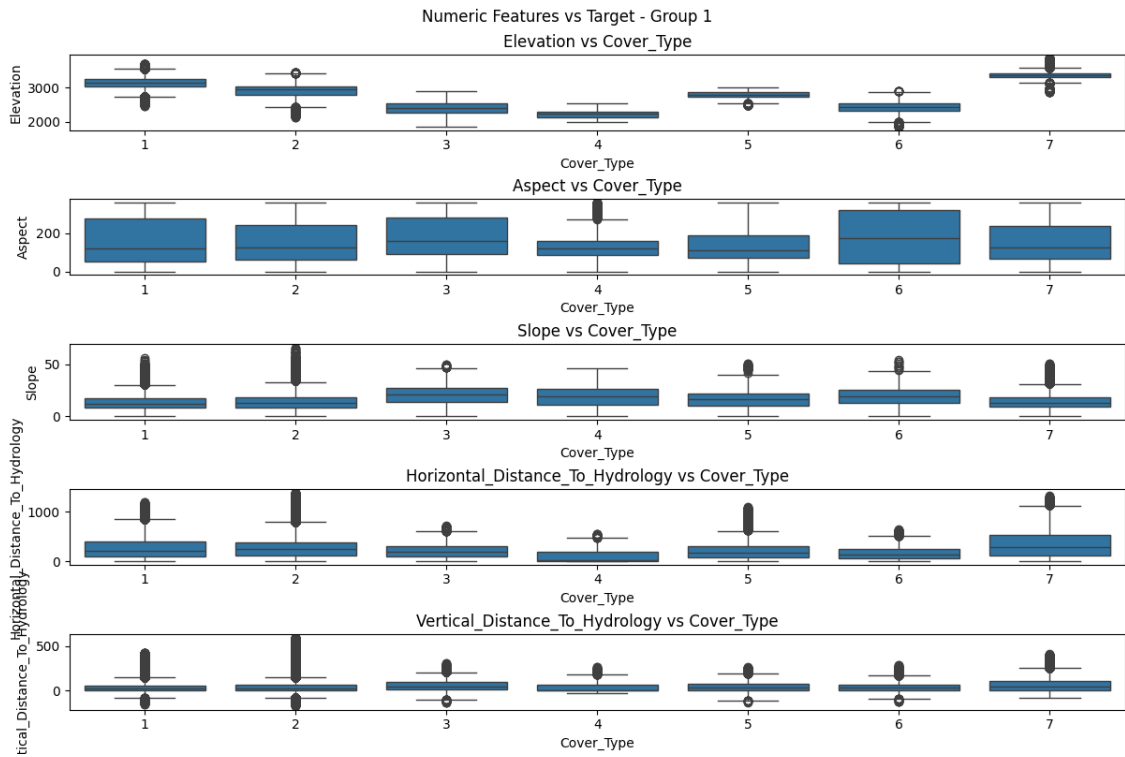
Categorical Features - Group 3





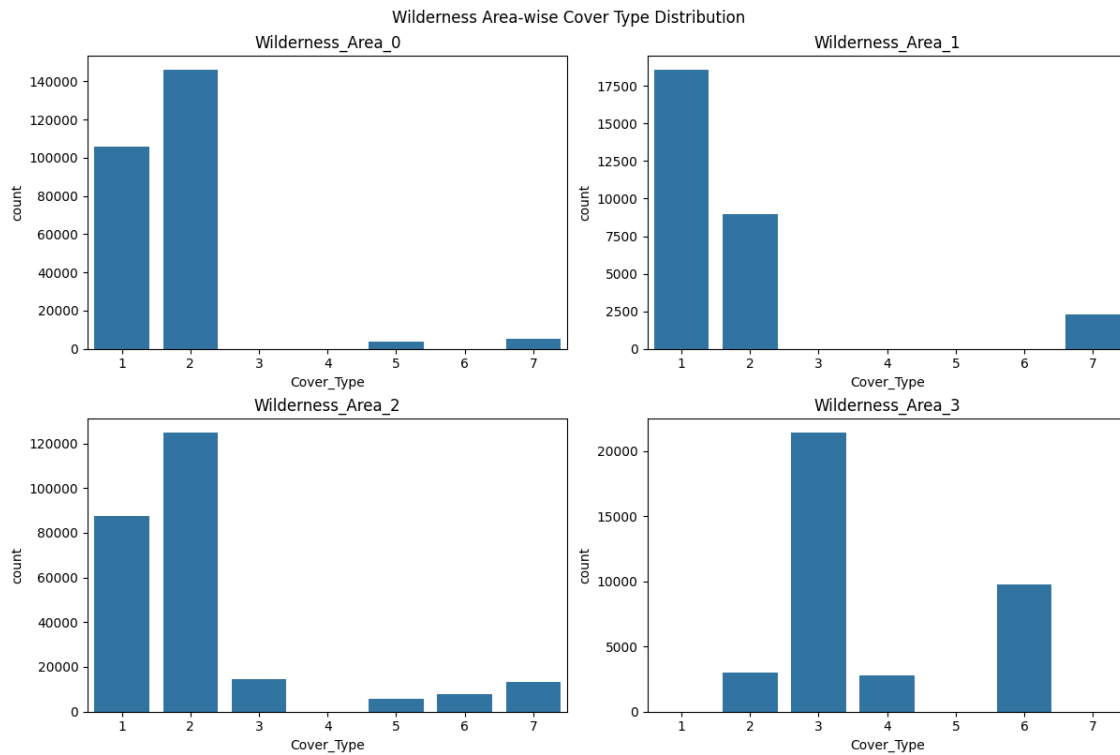
1.2.6 Numeric features vs Target

```
[5]: figs = eda.plot_numeric_vs_target()
    for fig in figs:
        fig.show()
```



1.2.7 Target distribution by wilderness area

```
[6]: eda.plot_cover_type_distribution_by_wilderness_area()  
plt.show()
```



1.3 Models

1.3.1 Logistic regression

Sigmoid

```
[5]: # dataset  
X, y = make_classification(  
    n_samples=500_000,  
    n_features=10,  
    n_informative=6,  
    n_classes=2,  
    random_state=42  
)  
  
# split  
X_train, X_temp, y_train, y_temp = train_test_split(  
    X, y, test_size=0.3, random_state=42  
)
```

```

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)

```

```

[4]: # model
model = SigmoidClassifier(
    max_iter=20,
    step_size=4,
    batch_size=1024
)

# fit
model.fit(X_train, y_train, X_val=X_val, y_val=y_val)

# accuracy
y_pred = model.predict(X_test)
acc = accuracy_score(y_test, y_pred)
print(f"Acc = {acc:.4f}")

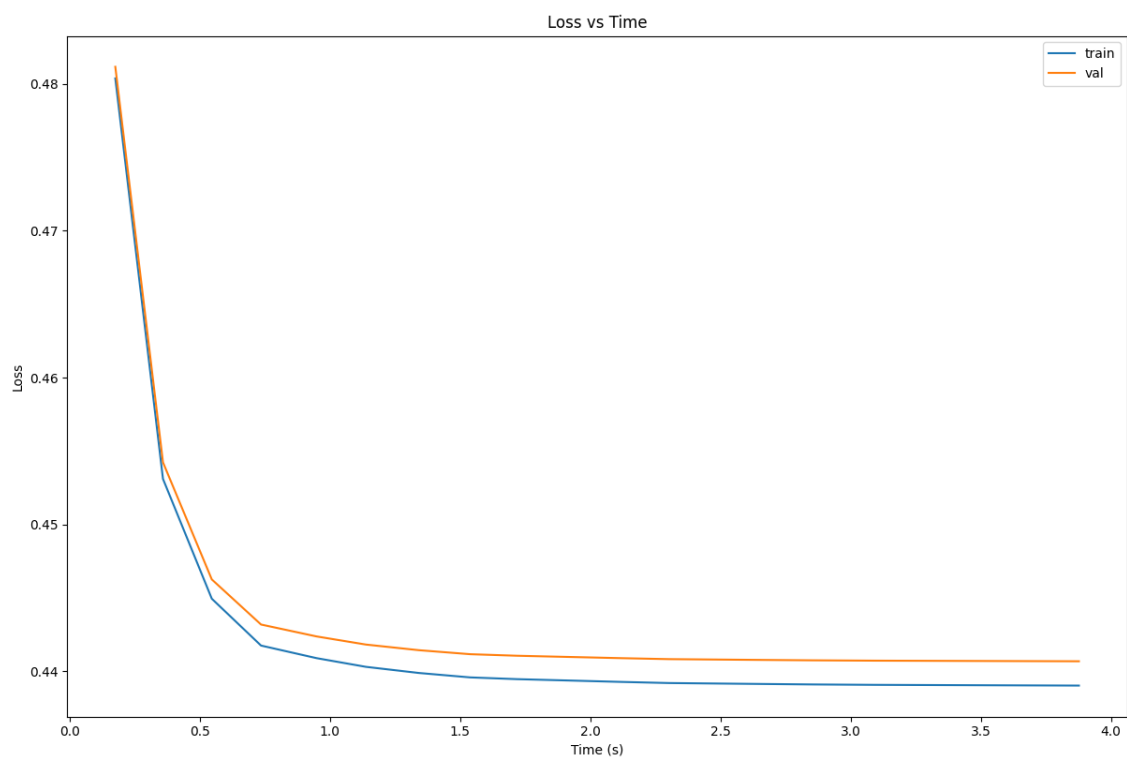
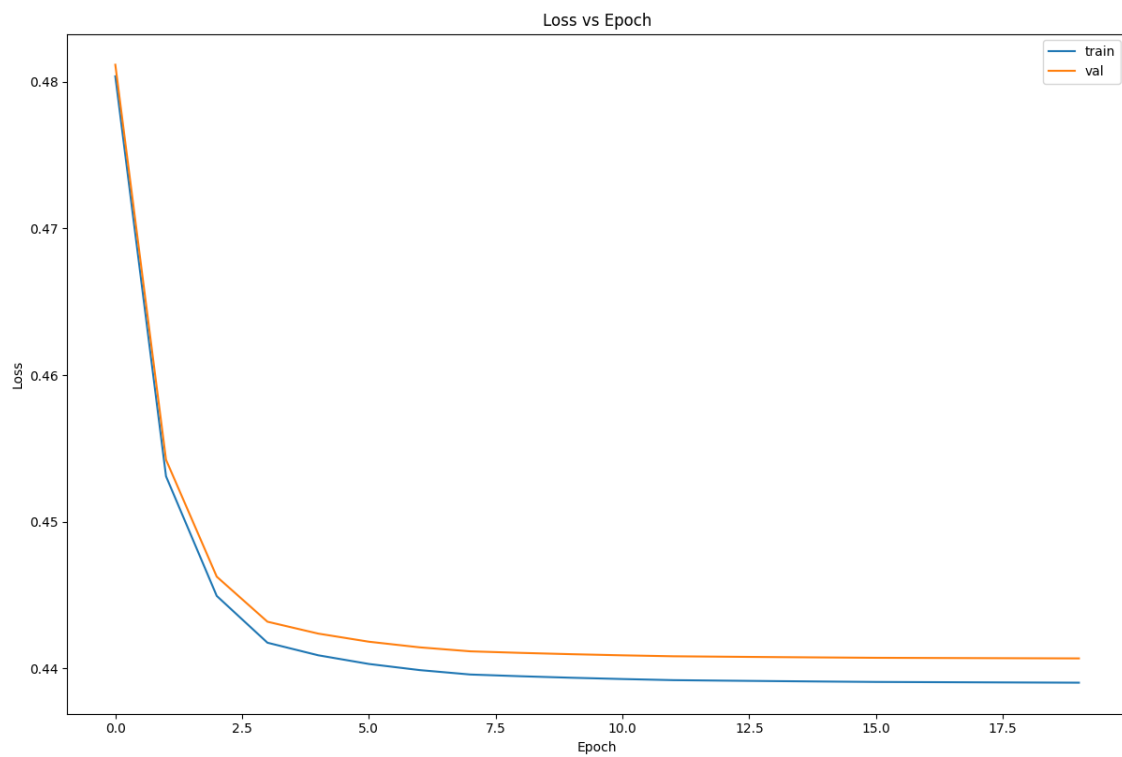
# ===== PLOT =====
epochs = range(len(model.train_loss_history_))

# 1. loss vs epoch
plt.figure(layout="constrained")
plt.plot(epochs, model.train_loss_history_, label="train")
plt.plot(epochs, model.val_loss_history_, label="val")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss vs Epoch")
plt.legend()
plt.show()

# 2. loss vs time
plt.figure(layout="constrained")
plt.plot(model.time_history_, model.train_loss_history_, label="train")
plt.plot(model.time_history_, model.val_loss_history_, label="val")
plt.xlabel("Time (s)")
plt.ylabel("Loss")
plt.title("Loss vs Time")
plt.legend()
plt.show()

```

Acc = 0.7983



Newton-Raphson IRLS

```
[4]: # model
model = IRLSClassifier()

# fit
model.fit(X_train, y_train, X_val=X_val, y_val=y_val)

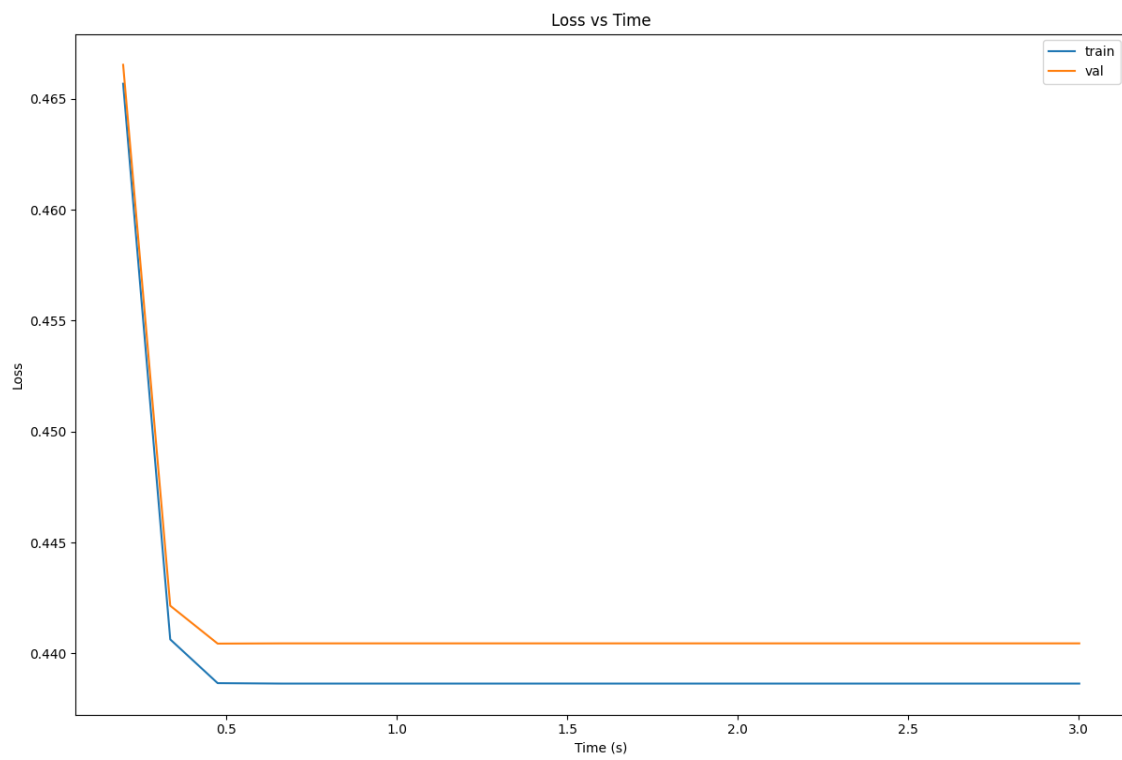
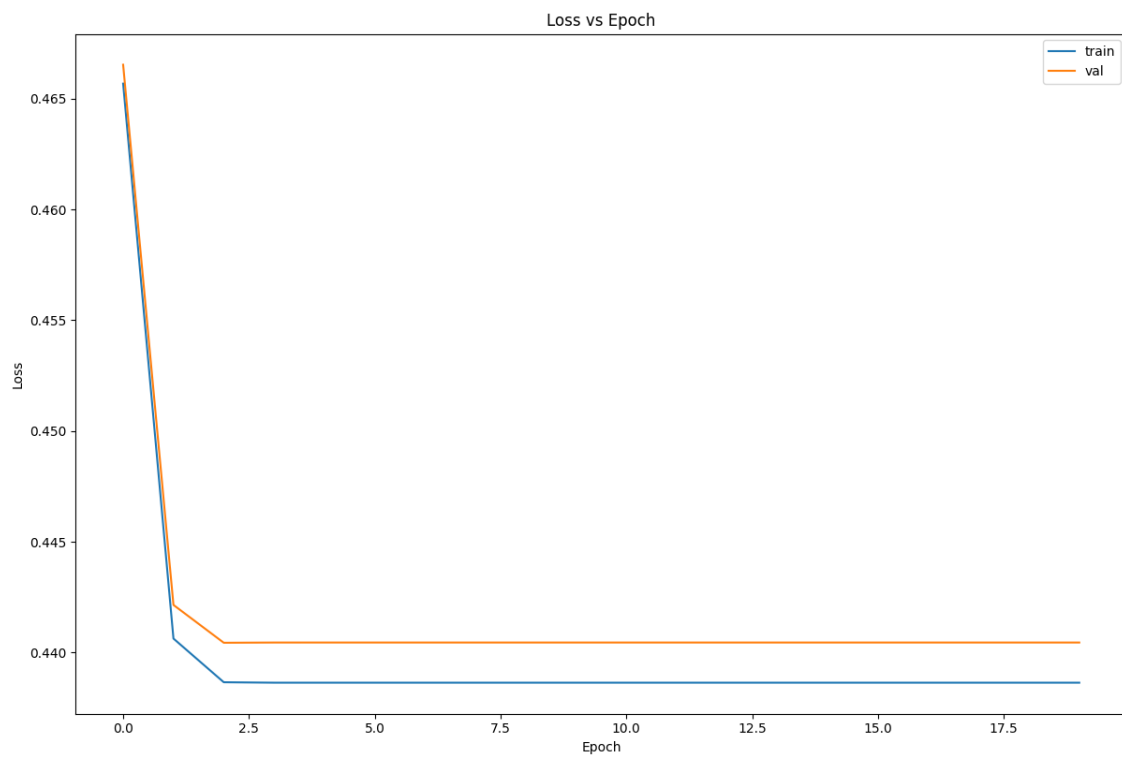
# accuracy
y_pred = model.predict(X_test)
acc = accuracy_score(y_test, y_pred)
print(f"Acc = {acc:.4f}")

# ===== PLOT =====
epochs = range(len(model.train_loss_history_))

# 1. loss vs epoch
plt.figure(layout="constrained")
plt.plot(epochs, model.train_loss_history_, label="train")
plt.plot(epochs, model.val_loss_history_, label="val")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss vs Epoch")
plt.legend()
plt.show()

# 2. loss vs time
plt.figure(layout="constrained")
plt.plot(model.time_history_, model.train_loss_history_, label="train")
plt.plot(model.time_history_, model.val_loss_history_, label="val")
plt.xlabel("Time (s)")
plt.ylabel("Loss")
plt.title("Loss vs Time")
plt.legend()
plt.show()
```

Acc = 0.7989



Softmax

```
[10]: softmax = get_pipeline(SoftmaxClassifier())
softmax.fit(d.X_train, d.y_train)
y_pred = softmax.predict(d.X_test)
y_true = d.y_test
acc = accuracy_score(y_true, y_pred)
print(f"Acc = {acc:.4f}")
```

Acc = 0.7148

OvR, OvO and Softmax

```
[7]: models = {
    "OvR Sigmoid": get_pipeline(OneVsRestClassifier(SigmoidClassifier())),
    "OvO Sigmoid": get_pipeline(OneVsOneClassifier(SigmoidClassifier())),
    "Softmax": get_pipeline(SoftmaxClassifier()),
}
```

```
[11]: evaluator = Evaluator()
result = evaluator.compare_models_test(
    models,
    d.X_train,
    d.y_train,
    d.X_test,
    d.y_test
)

# result.round(4).to_csv("ovr-ovo-softmax-compare-test.csv", index=False)

display(result)
```

	Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)	\
0	OvR Sigmoid	0.707323	0.574291	0.395866	0.386432	
1	OvO Sigmoid	0.716186	0.564049	0.424893	0.418528	
2	Softmax	0.714801	0.590134	0.430559	0.431943	

	ROC AUC	Precision_per_class \
0	None	[0.7014958151338425, 0.7339107713152038, 0.599...
1	None	[0.712009120236738, 0.7416016009448199, 0.6079...
2	None	[0.7115056748989235, 0.7407516532442873, 0.609...

	Recall_per_class \
0	[0.6785309667673716, 0.8030533003882809, 0.888...
1	[0.6928342145015106, 0.7979350511824921, 0.887...
2	[0.6895062311178247, 0.7986939639957642, 0.879...

	F1_per_class \
0	[0.6898223134050798, 0.7669267980245749, 0.716...
1	[0.7022908068664394, 0.7687376725838264, 0.721...

```
2 [0.7003332294488529, 0.768632379917114, 0.7196...
```

Confusion Matrix

```
0 [[28748, 12791, 43, 0, 5, 0, 781], [9618, 4550...
1 [[29354, 12121, 43, 0, 0, 0, 850], [9889, 4521...
2 [[29213, 12181, 43, 0, 2, 0, 929], [9793, 4525...
```

```
[9]: def plot_precision_and_confusion(result_df, class_names=None):
    """
    Returns
    -----
    fig : matplotlib.figure.Figure
    """

    n_models = len(result_df)

    first_prec = np.array(result_df.iloc[0]["Precision_per_class"])
    n_classes = len(first_prec)

    if class_names is None:
        class_names = [f"Class {i}" for i in range(n_classes)]

    fig, axes = plt.subplots(
        2, n_models,
        figsize=(6 * n_models, 10)
    )

    # ===== FIG TITLE =====
    fig.suptitle(
        "Model Comparison: Precision per Class & Confusion Matrix",
        fontsize=16
    )

    # fix shape when only 1 model
    if n_models == 1:
        axes = np.array([[axes[0]], [axes[1]]])

    for i, row in result_df.iterrows():
        model_name = row["Model"]

        precision = np.array(row["Precision_per_class"])
        cm = np.array(row["Confusion Matrix"])

        # ===== PRECISION =====
        ax1 = axes[0, i]
        ax1.bar(range(n_classes), precision)
        ax1.set_title(f"{model_name}\nPrecision per Class")
```

```

ax1.set_xlabel("Class")
ax1.set_ylabel("Precision")
ax1.set_xticks(range(n_classes))
ax1.set_xticklabels(class_names)
ax1.set_ylim(0, 1)

# ===== CONFUSION MATRIX =====
ax2 = axes[1, i]
im = ax2.imshow(cm, interpolation="nearest", cmap="Blues")

ax2.set_title(f"{model_name}\nConfusion Matrix")
ax2.set_xlabel("Predicted")
ax2.set_ylabel("True")

ax2.set_xticks(range(n_classes))
ax2.set_yticks(range(n_classes))
ax2.set_xticklabels(class_names)
ax2.set_yticklabels(class_names)

# annotate values
for r in range(n_classes):
    for c in range(n_classes):
        ax2.text(c, r, str(cm[r, c]),
                  ha="center", va="center")

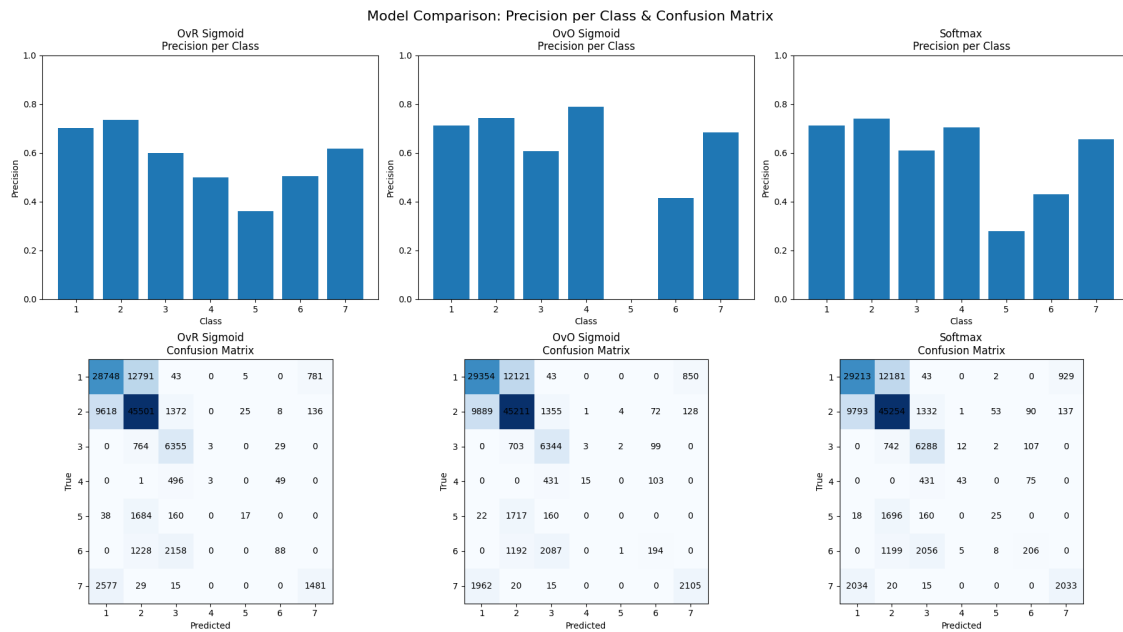
return fig

```

```

[10]: plot_precision_and_confusion(result, d.classes)
      plt.show()

```



1.3.2 LDA & QDA

LDA

```
[12]: model = get_pipeline(LDA(n_components=2))
      model.fit(d.X_train, d.y_train)
```

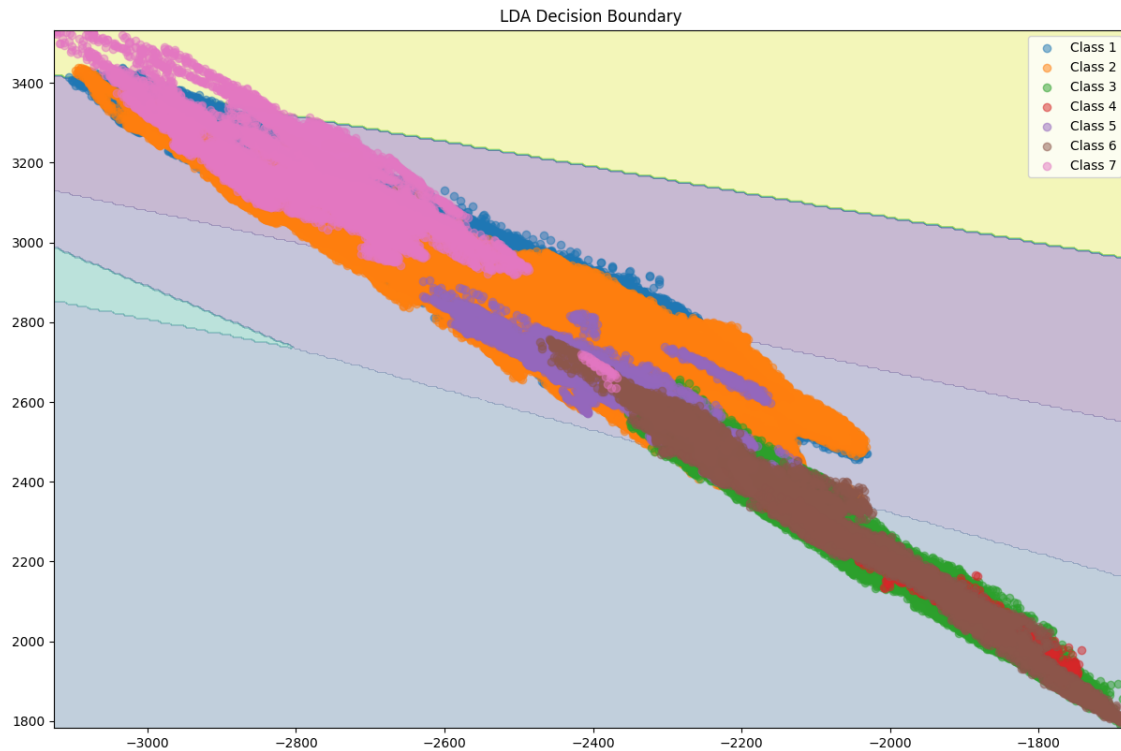
```
[12]: Pipeline(steps=[('features', None), ('scaler', StandardScaler()),
                      ('predictor', LDA(n_components=2))])
```

```
[13]: y_pred = model.predict(d.X_test)
      y_true = d.y_test
      acc = accuracy_score(y_true, y_pred)
      print(f"Acc = {acc:.4f}")
```

Acc = 0.6815

Decision Boundary 2D

```
[14]: model.named_steps["predictor"].plot2D(d.X_train, d.y_train)
```



QDA

```
[15]: model = get_pipeline(QDA())
      model.fit(d.X_train, d.y_train)
```

```
[15]: Pipeline(steps=[('features', None), ('scaler', StandardScaler()),
                      ('predictor', QDA())])
```

```
[16]: y_pred = model.predict(d.X_test)
      y_true = d.y_test
      acc = accuracy_score(y_true, y_pred)
      print(f"Acc = {acc:.4f}")
```

Acc = 0.2421

Fisher ratio

```
[17]: J_scores = fisher_ratio(d.X_train, d.y_train)
      df_ranking = pd.DataFrame({
          'Feature': d.feature_names,
          'Fisher_Ratio': J_scores
      }).sort_values(by='Fisher_Ratio', ascending=False).reset_index(drop=True)
      print(df_ranking)
```

	Feature	Fisher_Ratio
0	Elevation	1.604347

1	Wilderness_Area_3	1.183577
2	Soil_Type_9	0.288908
3	Soil_Type_3	0.132828
4	Soil_Type_37	0.129506
5	Soil_Type_2	0.129065
6	Soil_Type_38	0.120630
7	Wilderness_Area_0	0.106986
8	Soil_Type_1	0.106865
9	Horizontal_Distance_To_Roadways	0.106477
10	Soil_Type_5	0.086664
11	Slope	0.080869
12	Soil_Type_39	0.077892
13	Horizontal_Distance_To_Fire_Points	0.075265
14	Soil_Type_0	0.053114
15	Soil_Type_21	0.046517
16	Soil_Type_11	0.041572
17	Soil_Type_28	0.040414
18	Soil_Type_16	0.038440
19	Soil_Type_22	0.036609
20	Hillshade_9am	0.033738
21	Hillshade_Noon	0.030384
22	Soil_Type_4	0.026764
23	Horizontal_Distance_To_Hydrology	0.024378
24	Wilderness_Area_1	0.023078
25	Soil_Type_29	0.022497
26	Soil_Type_34	0.021798
27	Soil_Type_13	0.020662
28	Soil_Type_12	0.019182
29	Soil_Type_36	0.014480
30	Wilderness_Area_2	0.012739
31	Vertical_Distance_To_Hydrology	0.012515
32	Soil_Type_10	0.011990
33	Soil_Type_31	0.011726
34	Hillshade_3pm	0.010834
35	Soil_Type_32	0.009024
36	Soil_Type_23	0.007269
37	Soil_Type_30	0.006554
38	Aspect	0.004655
39	Soil_Type_17	0.003550
40	Soil_Type_25	0.002846
41	Soil_Type_18	0.002285
42	Soil_Type_20	0.002202
43	Soil_Type_19	0.001987
44	Soil_Type_33	0.001831
45	Soil_Type_15	0.001333
46	Soil_Type_27	0.001324
47	Soil_Type_35	0.001257
48	Soil_Type_8	0.001140

49	Soil_Type_26	0.000354
50	Soil_Type_24	0.000244
51	Soil_Type_6	0.000193
52	Soil_Type_7	0.000110
53	Soil_Type_14	0.000093

1.3.3 Perceptron & Regularization

Perceptron

```
[5]: # ===== 1. LINEAR DATA =====
result = make_blobs(n_samples=200, centers=2, random_state=42)
X_linear, y_linear = result[:2]
y_linear = np.where(y_linear == 0, -1, 1)

model1 = Perceptron(lr=0.01, epochs=50)
model1.fit(X_linear, y_linear)

# ===== 2. NON-LINEAR DATA =====
X_non, y_non = make_circles(n_samples=200, noise=0.1, factor=0.5)
y_non = np.where(y_non == 0, -1, 1)

model2 = Perceptron(lr=0.01, epochs=50)
model2.fit(X_non, y_non)

# ===== PLOT =====
plt.figure(figsize=(10, 8), constrained_layout=True)

plt.subplot(2, 2, 1)
plt.scatter(X_linear[:, 0], X_linear[:, 1], c=y_linear, s=5)
# draw decision boundary
x_vals = np.linspace(X_linear[:, 0].min(), X_linear[:, 0].max(), 100)
y_vals = -(model1.w[0] * x_vals + model1.b) / model1.w[1]
plt.plot(x_vals, y_vals, linewidth=2)
plt.title("Linear Separable Data (Blobs)")

plt.subplot(2, 2, 2)
plt.scatter(X_non[:, 0], X_non[:, 1], c=y_non, s=5)
x_vals = np.linspace(X_non[:, 0].min(), X_non[:, 0].max(), 100)
y_vals = -(model2.w[0] * x_vals + model2.b) / model2.w[1]
plt.plot(x_vals, y_vals, linewidth=2)
plt.title("Non-linear Data")

plt.subplot(2, 2, 3)
plt.plot(model1.errors_, marker='o', markersize=3, linewidth=1)
plt.title("Linear Data (Converges)")
plt.xlabel("Epoch")
plt.ylabel("Error rate")
plt.grid(alpha=0.3)
```

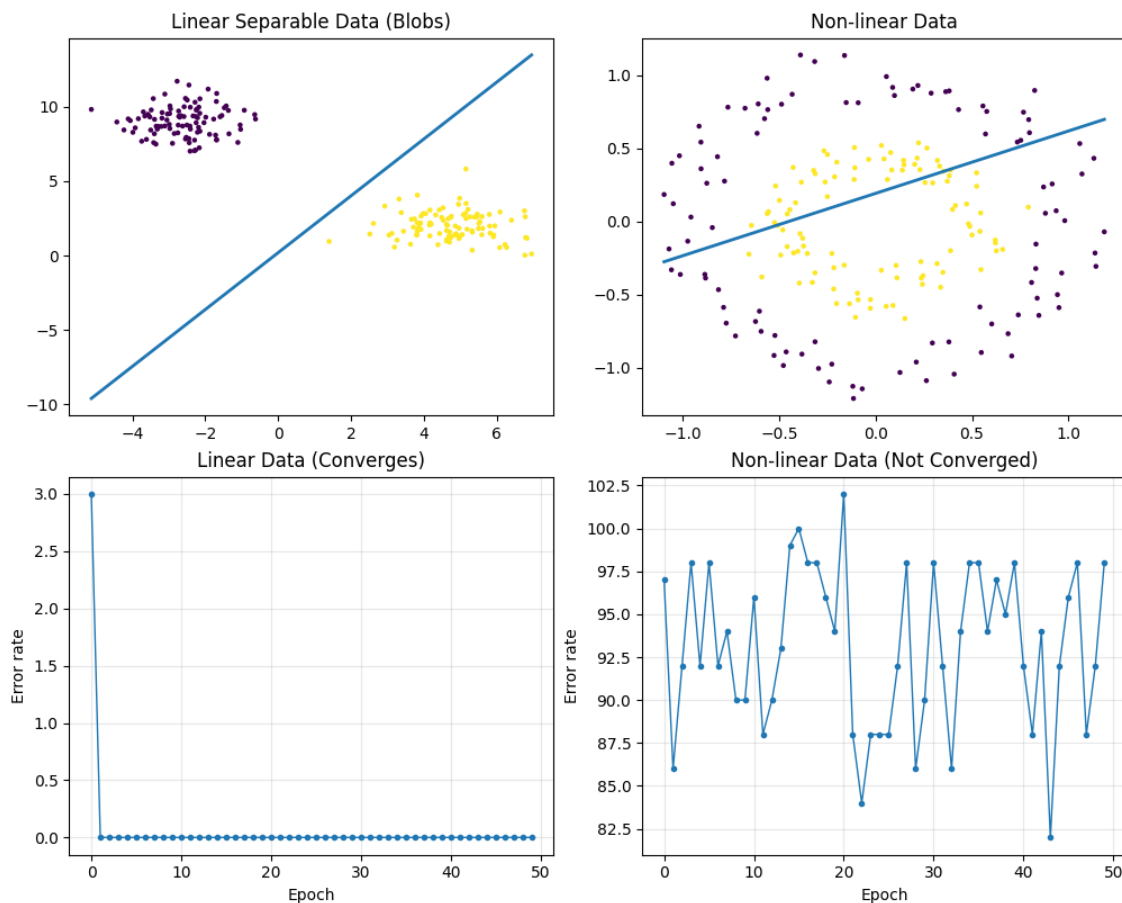
```

plt.subplot(2, 2, 4)
plt.plot(model2.errors_, marker='o', markersize=3, linewidth=1)
plt.title("Non-linear Data (Not Converged)")
plt.xlabel("Epoch")
plt.ylabel("Error rate")
plt.grid(alpha=0.3)

plt.show()

# ===== PRINT =====
print("Linear:", "Converged" if model1.errors_[-1] == 0 else "Not converged")
print("Non-linear:", "Converged" if model2.errors_[-1] == 0 else "Not
↪converged")

```



Linear: Converged
Non-linear: Not converged

Regularization

```
[8]: # ===== L1 Logistic Regression =====
model_l1 = LogisticRegression(
    solver='saga',
    l1_ratio=1.0,          # = L1
    C=0.01,
    max_iter=2000
)
model_l1.fit(X_train, y_train)

# ===== L2 Logistic Regression =====
model_l2 = LogisticRegression(
    solver='saga',
    l1_ratio=0.0,          # = L2
    C=1.0,
    max_iter=1000
)
model_l2.fit(X_train, y_train)

# ===== Accuracy =====
acc_l1 = accuracy_score(y_test, model_l1.predict(X_test))
acc_l2 = accuracy_score(y_test, model_l2.predict(X_test))

# ===== Sparsity =====
w_l1 = model_l1.coef_
w_l2 = model_l2.coef_

sparsity_l1 = np.mean(np.abs(w_l1) < 1e-6)
sparsity_l2 = np.mean(np.abs(w_l2) < 1e-6)

# ===== PRINT =====
print("=== RESULTS ===")
print(f"L1 Accuracy: {acc_l1:.4f}")
print(f"L2 Accuracy: {acc_l2:.4f}")
print(f"L1 Sparsity: {sparsity_l1:.4f}")
print(f"L2 Sparsity: {sparsity_l2:.4f}")
```

```
=== RESULTS ===
L1 Accuracy: 0.7988
L2 Accuracy: 0.7989
L1 Sparsity: 0.3000
L2 Sparsity: 0.0000
```

Stratified k-fold CV

```
[3]: X = d.X
y = d.y
print("Dataset shape:", X.shape)
print("Class distribution:", np.bincount(y))
```

```
# ===== Lambda=====
lambdas = [0.001, 0.01, 0.1, 1, 10]

# ===== Function choose best lambda =====
best_lambda, best_score = select_best_lambda(X, y, lambdas)

print("\n=== RESULT ===")
print("Best lambda:", best_lambda)
print("Best log loss:", best_score)
```

```
Dataset shape: (581012, 54)
Class distribution: [    0 211840 283301 35754 2747 9493 17367 20510]
Lambda = 0.001, Log Loss = 0.6290
Lambda = 0.01, Log Loss = 0.6290
Lambda = 0.1, Log Loss = 0.6290
Lambda = 1, Log Loss = 0.6290
Lambda = 10, Log Loss = 0.6292

=== RESULT ===
Best lambda: 0.01
Best log loss: 0.6289983569404619
```

Class-weighted loss

```
[6]: # ===== SCALE DATA (RẤT QUAN TRỌNG) =====
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ===== 1. Logistic Regression (baseline - KHÔNG weight) =====
print("===== LOGISTIC REGRESSION (NO CLASS WEIGHT) =====")

logistic_model = LogisticRegression(
    max_iter=2000,      # tăng để hội tụ
)

logistic_model.fit(X_train_scaled, y_train)
y_pred_lr = logistic_model.predict(X_test_scaled)

print("Accuracy:", accuracy_score(y_test, y_pred_lr))
print(classification_report(y_test, y_pred_lr, digits=4, zero_division=0))

# ===== 2. Logistic Regression (CÓ class_weight) =====
print("\n===== LOGISTIC REGRESSION (BALANCED) =====")
```

```

logistic_balanced = LogisticRegression(
    max_iter=2000,
    class_weight='balanced',
)

logistic_balanced.fit(X_train_scaled, y_train)
y_pred_bal = logistic_balanced.predict(X_test_scaled)

print("Accuracy:", accuracy_score(y_test, y_pred_bal))
print(classification_report(y_test, y_pred_bal, digits=4, zero_division=0))

# ===== 3. Weighted Softmax (custom) =====
print("\n===== WEIGHTED SOFTMAX (CUSTOM) =====")

model_weight = WeightedSoftmaxClassifier(
    lr=0.05,
    max_iter=500,
    batch_size=2048    # giúp ổn định hơn với dataset lớn
)

model_weight.fit(X_train_scaled, y_train)
y_pred_w = model_weight.predict(X_test_scaled)

print("Accuracy:", accuracy_score(y_test, y_pred_w))
print(classification_report(y_test, y_pred_w, digits=4, zero_division=0))

# ===== In class weights =====
print("\n===== CLASS WEIGHTS (CUSTOM MODEL) =====")
for i, w in enumerate(model_weight.class_weights_):
    print(f"Class {model_weight.classes_[i]}: weight = {w:.4f}")

```

===== LOGISTIC REGRESSION (NO CLASS WEIGHT) =====

Accuracy: 0.7989466666666667

	precision	recall	f1-score	support
0	0.8042	0.7927	0.7984	37670
1	0.7938	0.8053	0.7995	37330
accuracy			0.7989	75000
macro avg	0.7990	0.7990	0.7989	75000
weighted avg	0.7990	0.7989	0.7989	75000

===== LOGISTIC REGRESSION (BALANCED) =====

Accuracy: 0.79892

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.8040	0.7930	0.7985	37670
1	0.7940	0.8048	0.7994	37330
accuracy			0.7989	75000
macro avg	0.7990	0.7989	0.7989	75000
weighted avg	0.7990	0.7989	0.7989	75000

===== WEIGHTED SOFTMAX (CUSTOM) =====

Accuracy: 0.7989333333333334

	precision	recall	f1-score	support
0	0.8040	0.7930	0.7985	37670
1	0.7940	0.8049	0.7994	37330
accuracy			0.7989	75000
macro avg	0.7990	0.7990	0.7989	75000
weighted avg	0.7990	0.7989	0.7989	75000

===== CLASS WEIGHTS (CUSTOM MODEL) =====

Class 0: weight = 2.0023

Class 1: weight = 1.9977

1.4 Advanced

1.4.1 Probit

```
[19]: class Data:
    def __init__(self, X, y, test_size=0.3, random_state=42):
        self.X = X
        self.y = y
        X_train, X_test, y_train, y_test = train_test_split(
            X, y,
            test_size=test_size,
            random_state=random_state,
            stratify=y
        )
        self.X_train = X_train
        self.X_test = X_test
        self.y_train = y_train
        self.y_test = y_test

    def make_classification_data(test_size=0.3, random_state=42):
        # ===== generate blobs =====
        centers = [[-2, 0], [0, 1.5]]
```

```

X, y = make_blobs(
    n_samples=1_000,
    centers=centers,
    cluster_std=1,
    random_state=random_state
)

# ===== linear transformation (tạo correlation + rotate) =====
transformation = [[0.4, 0.2], [-0.2, 0.5]]
X = np.dot(X, transformation)

return Data(X, y, test_size, random_state)

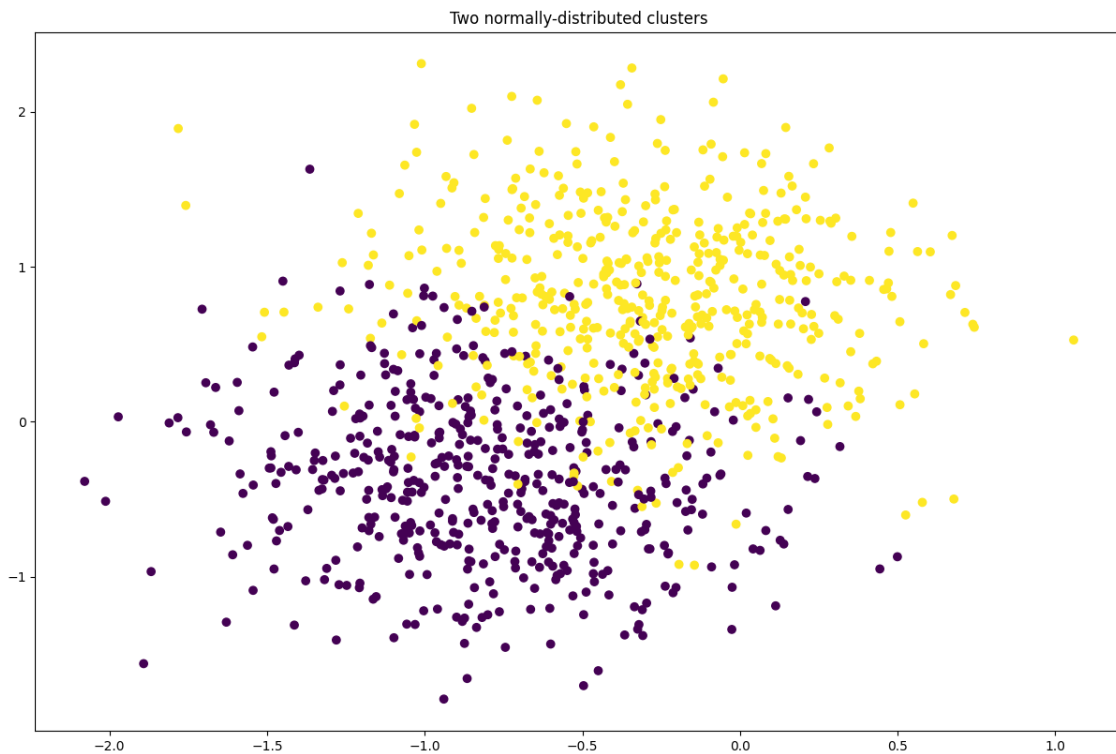
bd = make_classification_data()

```

```

[20]: plt.figure()
plt.scatter(bd.X[:, 0], bd.X[:, 1], c=bd.y)
plt.title("Two normally-distributed clusters")
plt.show()

```



```

[21]: def plot_decision_boundary_2d(
    model, X, y, resolution=200, title="Decision Boundary"
):

```

```

import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots()

x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, resolution),
    np.linspace(y_min, y_max, resolution)
)

# grid trực tiếp 2D (không cần mean nữa)
grid = np.c_[xx.ravel(), yy.ravel()]

Z = model.predict(grid).reshape(xx.shape)

ax.contourf(xx, yy, Z, alpha=0.3, cmap="coolwarm")

ax.scatter(
    X[:, 0], X[:, 1],
    c=y, s=20, edgecolor="k", cmap="coolwarm"
)

ax.set_xlabel("Feature 0")
ax.set_ylabel("Feature 1")
ax.set_title(title)

return fig

def plot_probability_2d(
    model, X, y, class_idx=1,
    resolution=200, title="Probability"
):
    import matplotlib.pyplot as plt
    import numpy as np

    fig, ax = plt.subplots()

    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, resolution),
        np.linspace(y_min, y_max, resolution)
    )

```

```

grid = np.c_[xx.ravel(), yy.ravel()]

Z = model.predict_proba(grid)[:, class_idx].reshape(xx.shape)

contour = ax.contourf(xx, yy, Z, levels=50, cmap="viridis")

ax.scatter(
    X[:, 0], X[:, 1],
    c=y, s=20, edgecolor="k", cmap="viridis"
)

fig.colorbar(contour, ax=ax, label=f"p(class={class_idx})")

ax.set_xlabel("Feature 0")
ax.set_ylabel("Feature 1")
ax.set_title(title)

return fig

```

```

[24]: sigmoid = SigmoidClassifier(batch_size=32, max_iter=500, step_size=100)
      sigmoid.fit(bd.X, bd.y)

      probit = ProbitClassifier(batch_size=32, max_iter=500, step_size=100)
      probit.fit(bd.X, bd.y)

      sgd = SGDClassifier(random_state=42)
      sgd.fit(bd.X, bd.y)

```

```

[24]: SGDClassifier(random_state=42)

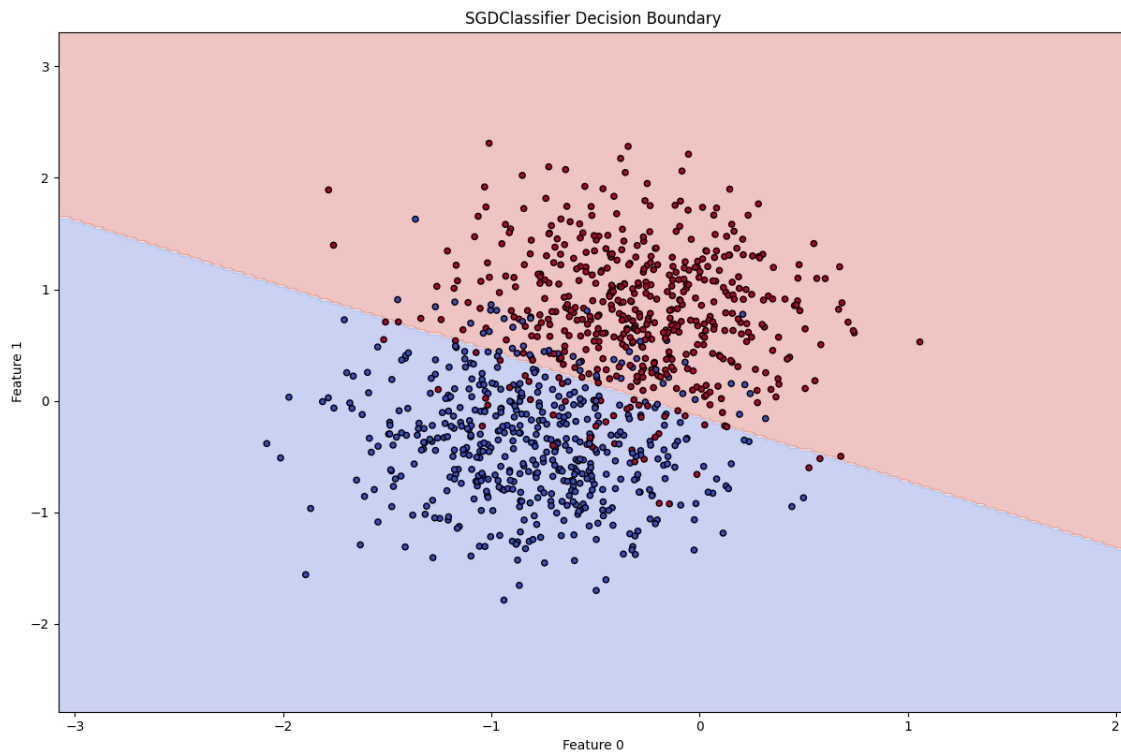
```

Decision boundary

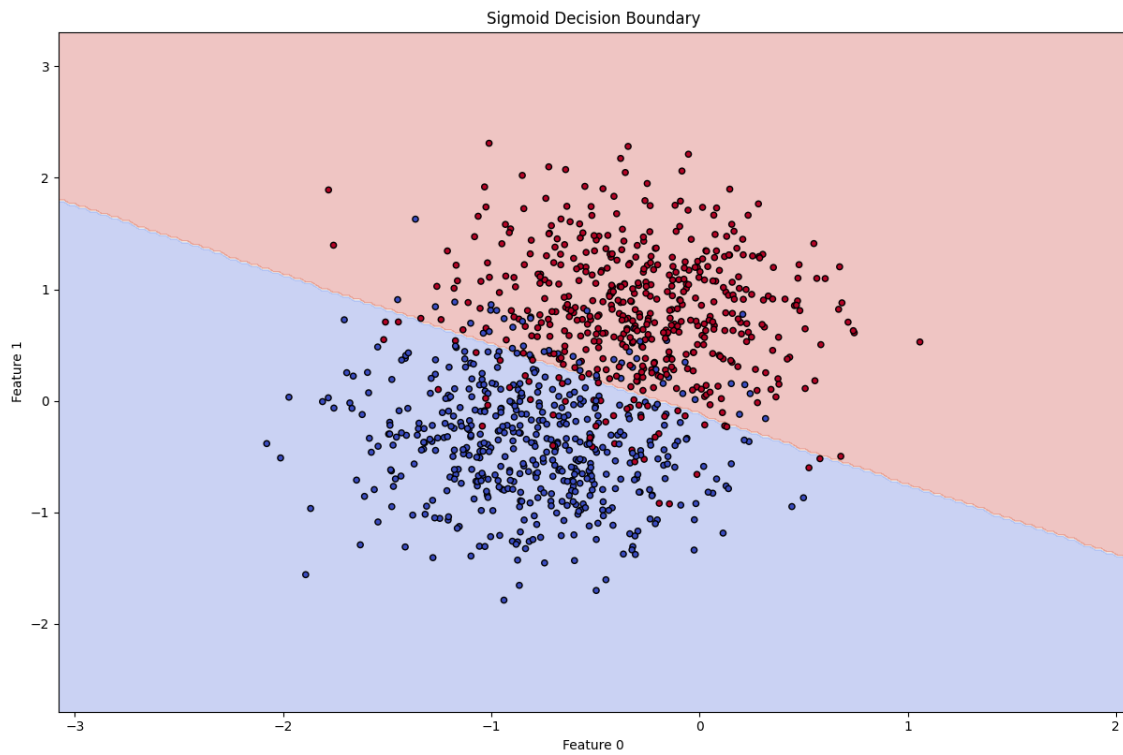
```

[25]: plot_decision_boundary_2d(sgd, bd.X, bd.y, title="SGDClassifier Decision_
      ↪Boundary")
      plt.show()

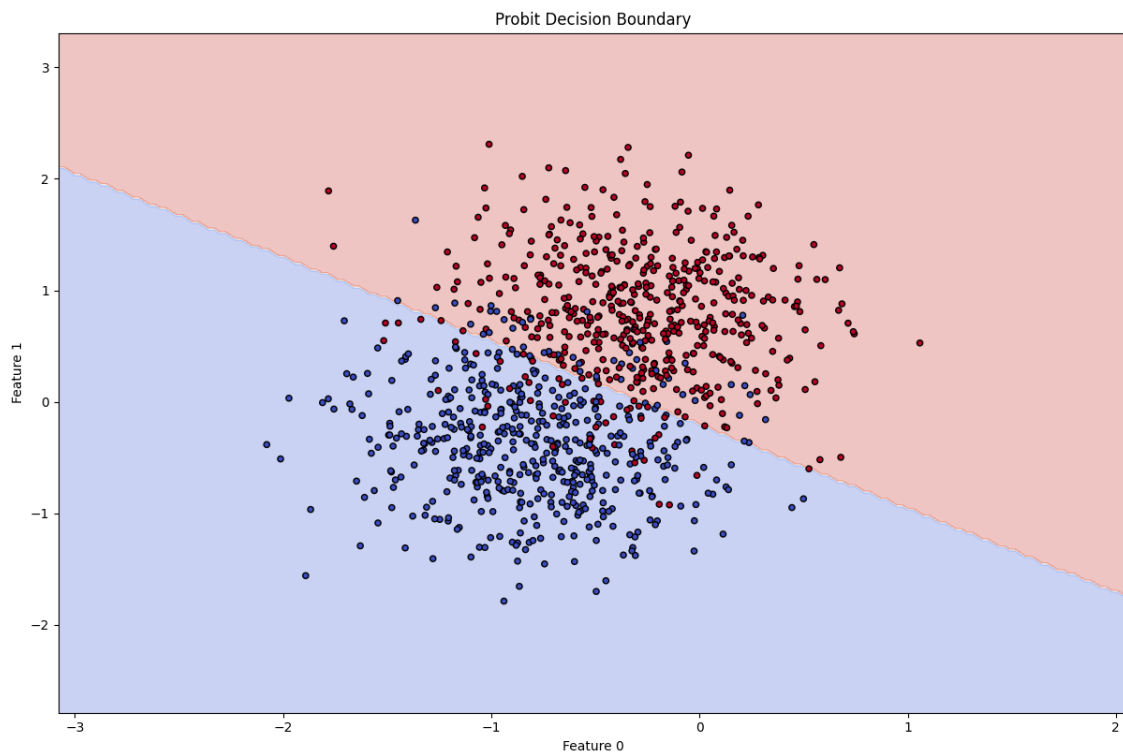
```



```
[26]: plot_decision_boundary_2d(sigmoid, bd.X, bd.y, title="Sigmoid Decision_□  
      ↪Boundary")  
      plt.show()
```

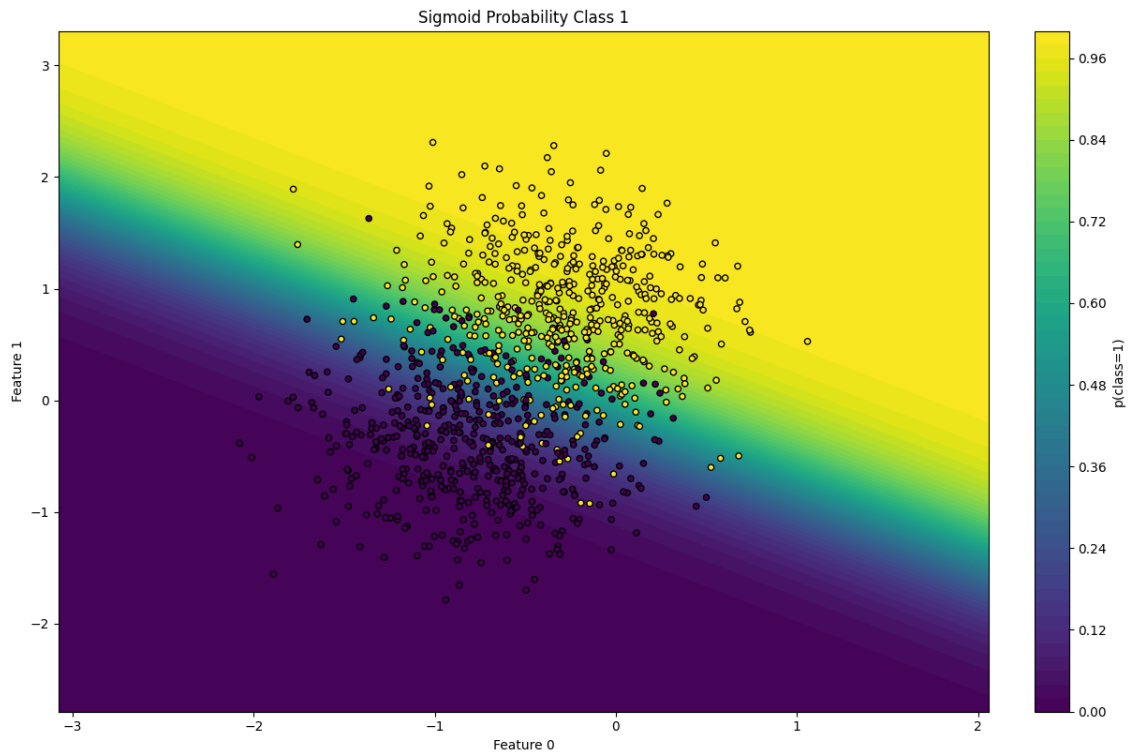


```
[27]: plot_decision_boundary_2d(probit, bd.X, bd.y, title="Probit Decision Boundary")  
plt.show()
```

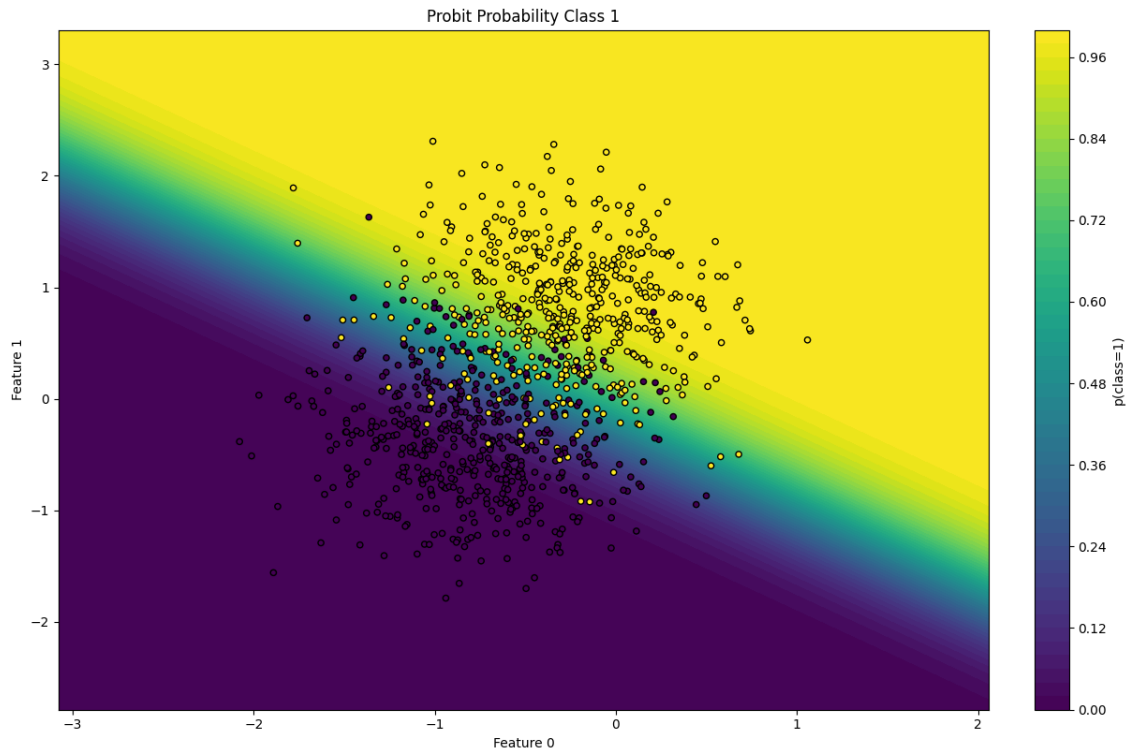


Predict probability

```
[28]: plot_probability_2d(sigmoid, bd.X, bd.y, class_idx=1, title="Sigmoid_␣  
      ↪Probability Class 1")  
      plt.show()
```



```
[29]: plot_probability_2d(probit, bd.X, bd.y, class_idx=1, title="Probit Probability_␣  
      ↪Class 1")  
      plt.show()
```



Noise sensitivity

```
[30]: def compute_cls_metrics(y_true, y_pred, y_proba=None):
    acc = accuracy_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred, average="macro")
    prec = precision_score(y_true, y_pred, average="macro", zero_division=0)
    rec = recall_score(y_true, y_pred, average="macro", zero_division=0)

    metrics = (acc, f1, prec, rec)

    if y_proba is not None:
        try:
            auc = roc_auc_score(y_true, y_proba)
        except:
            auc = np.nan
        metrics = metrics + (auc,)
    else:
        metrics = metrics + (np.nan,)

    return metrics

def add_boundary_noise_sgd(X, y, noise_ratio, alpha=2, seed=42):
    rng = np.random.default_rng(seed)
```

```

sgd = SGDClassifier(loss="log_loss", random_state=seed)
sgd.fit(X, y)

logits = sgd.decision_function(X)
margin = np.abs(logits)

p_flip = np.exp(-alpha * margin)
p_flip = p_flip / (p_flip.max() + 1e-8)

flip_mask = rng.random(len(y)) < noise_ratio * p_flip

y_noisy = y.copy()
y_noisy[flip_mask] = 1 - y_noisy[flip_mask]

return y_noisy

def label_noise_sensitivity_df(models, data, noise_ratio=0.1, seed=42):
    # rng = np.random.default_rng(seed)
    rows = []

    classes = np.unique(data.y_train)

    # ===== CLEAN TRAIN =====
    clean_scores = {}

    for name, model in models.items():
        m = clone(model)
        m.fit(data.X_train, data.y_train)

        y_pred = m.predict(data.X_test)

        y_proba = None
        if hasattr(m, "predict_proba"):
            y_proba = m.predict_proba(data.X_test)
            if y_proba.shape[1] == 2:
                y_proba = y_proba[:, 1]

        clean_scores[name] = compute_cls_metrics(
            data.y_test, y_pred, y_proba
        )

    # ===== ADD LABEL NOISE =====
    # y_train_noisy = data.y_train.copy()

    # n_flip = int(len(y_train_noisy) * noise_ratio)
    # flip_idx = rng.choice(len(y_train_noisy), n_flip, replace=False)

```

```

# if len(classes) == 2:
#     y_train_noisy[flip_idx] = 1 - y_train_noisy[flip_idx]
# else:
#     y_train_noisy[flip_idx] = rng.choice(classes, size=n_flip)

y_train_noisy = add_boundary_noise_sgd(data.X_train, data.y_train,
↪noise_ratio, seed=seed)

# ===== NOISY TRAIN =====
noisy_scores = {}

for name, model in models.items():
    m = clone(model)
    m.fit(data.X_train, y_train_noisy)

    y_pred = m.predict(data.X_test)

    y_proba = None
    if hasattr(m, "predict_proba"):
        y_proba = m.predict_proba(data.X_test)
        if y_proba.shape[1] == 2:
            y_proba = y_proba[:, 1]

    noisy_scores[name] = compute_cls_metrics(
        data.y_test, y_pred, y_proba
    )

# ===== BUILD DATAFRAME =====
for name in models.keys():
    (cacc, cf1, cprec, crec, cauc) = clean_scores[name]
    (nacc, nf1, nprec, nrec, nauc) = noisy_scores[name]

    rows.append({
        "Model": name,

        # Accuracy
        "Acc_clean": cacc,
        "Acc_noisy": nacc,
        "Acc_sensitivity": (cacc - nacc) / (cacc + 1e-8),

        # F1
        "F1_clean": cf1,
        "F1_noisy": nf1,
        "F1_sensitivity": (cf1 - nf1) / (cf1 + 1e-8),

        # Precision
        "Precision_clean": cprec,

```

```

        "Precision_noisy": nprec,
        "Precision_sensitivity": (cprec - nprec) / (cprec + 1e-8),

        # Recall
        "Recall_clean": crec,
        "Recall_noisy": nrec,
        "Recall_sensitivity": (crec - nrec) / (crec + 1e-8),

        # AUC
        "AUC_clean": cauc,
        "AUC_noisy": nauc,
        "AUC_sensitivity": (cauc - nauc) if not np.isnan(cauc) else np.nan,
    })

    return pd.DataFrame(rows)

```

```
[31]: df = label_noise_sensitivity_df(models, bd, 0.1)
```

```
display(df)
```

	Model	Acc_clean	Acc_noisy	Acc_sensitivity	F1_clean	F1_noisy	\
0	Sigmoid	0.903333	0.903333	0.0	0.903281	0.903281	
1	Probit	0.903333	0.903333	0.0	0.903281	0.903281	

	F1_sensitivity	Precision_clean	Precision_noisy	Precision_sensitivity	\
0	0.0	0.904214	0.904214		0.0
1	0.0	0.904214	0.904214		0.0

	Recall_clean	Recall_noisy	Recall_sensitivity	AUC_clean	AUC_noisy	\
0	0.903333	0.903333	0.0	0.961111	0.961111	
1	0.903333	0.903333	0.0	0.961244	0.961200	

	AUC_sensitivity
0	0.000000
1	0.000044

```
[35]: models = {
        "Sigmoid": SigmoidClassifier(batch_size=32, max_iter=500, step_size=100),
        "Probit": ProbitClassifier(batch_size=32, max_iter=500, step_size=100),
    }

    df = label_noise_sensitivity_df(models, bd, 0.2)

    # df = df.round(4)
    # df.to_csv("probit-sigmoid-label-noise-sensitivity.csv", index=False)

    display(df)

```

	Model	Acc_clean	Acc_noisy	Acc_sensitivity	F1_clean	F1_noisy	\
0	Sigmoid	0.893333	0.89	0.003731	0.893291	0.889969	
1	Probit	0.900000	0.90	0.000000	0.899960	0.899960	

	F1_sensitivity	Precision_clean	Precision_noisy	Precision_sensitivity	\
0	0.003718	0.893964	0.890434	0.003949	
1	0.000000	0.900641	0.900641	0.000000	

	Recall_clean	Recall_noisy	Recall_sensitivity	AUC_clean	AUC_noisy	\
0	0.893333	0.89	0.003731	0.964178	0.964267	
1	0.900000	0.90	0.000000	0.963778	0.963867	

	AUC_sensitivity
0	-0.000089
1	-0.000089

```
[36]: df = label_noise_sensitivity_df(models, bd, 0.3)

display(df)
```

	Model	Acc_clean	Acc_noisy	Acc_sensitivity	F1_clean	F1_noisy	\
0	Sigmoid	0.893333	0.890000	0.003731	0.893291	0.889969	
1	Probit	0.900000	0.903333	-0.003704	0.899960	0.903332	

	F1_sensitivity	Precision_clean	Precision_noisy	Precision_sensitivity	\
0	0.003718	0.893964	0.890434	0.003949	
1	-0.003747	0.900641	0.903351	-0.003009	

	Recall_clean	Recall_noisy	Recall_sensitivity	AUC_clean	AUC_noisy	\
0	0.893333	0.890000	0.003731	0.964178	0.964311	
1	0.900000	0.903333	-0.003704	0.963778	0.963822	

	AUC_sensitivity
0	-0.000133
1	-0.000044

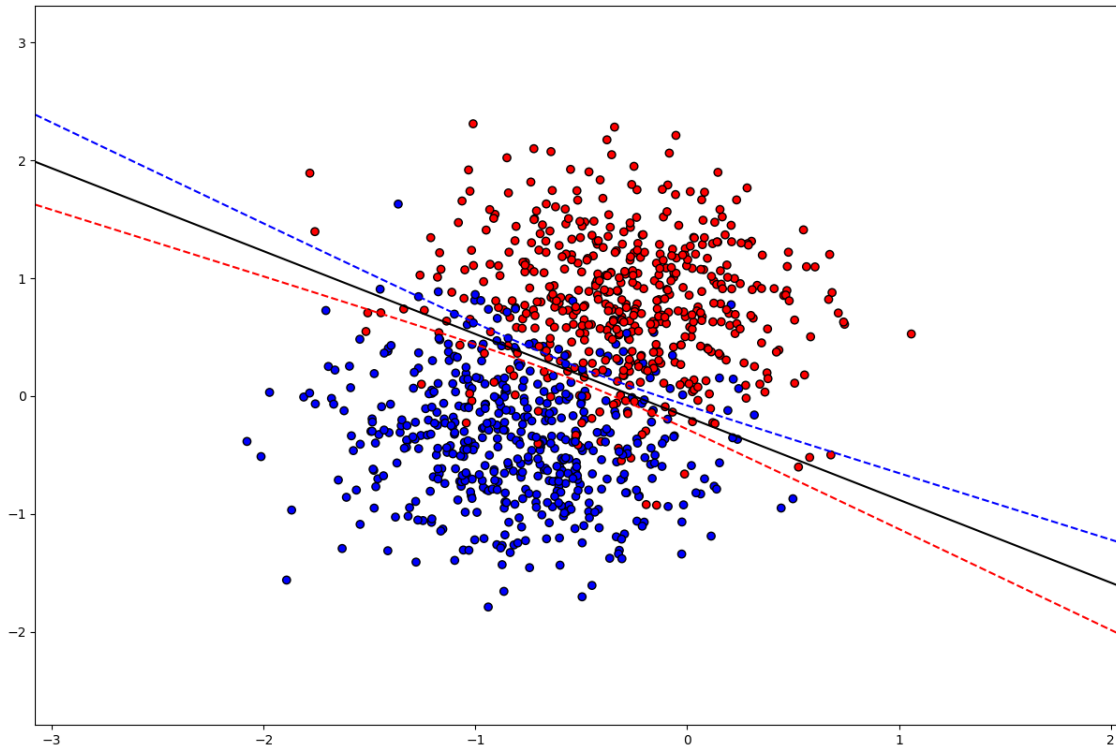
1.4.2 Laplace approximation

```
[34]: # ===== Train Sigmoid (Logistic) =====
sigmoid = SigmoidClassifier(batch_size=32, max_iter=1000, step_size=200)
sigmoid.fit(bd.X, bd.y)

# ===== Laplace Approximation =====
laplace = LaplaceApprox(sigmoid)
laplace.fit(bd.X)

# ===== Plot decision boundary + uncertainty =====
fig, ax = plt.subplots()
```

```
laplace.decision_boundary_sigma(bd.X, bd.y, ax=ax)
plt.show()
```



1.4.3 Kernel Logistic Regression

```
[13]: X, y = make_moons(n_samples=400, noise=0.25, random_state=42)
```

```
[14]: linear_model = LogisticRegression()
linear_model.fit(X, y)
```

```
[14]: LogisticRegression()
```

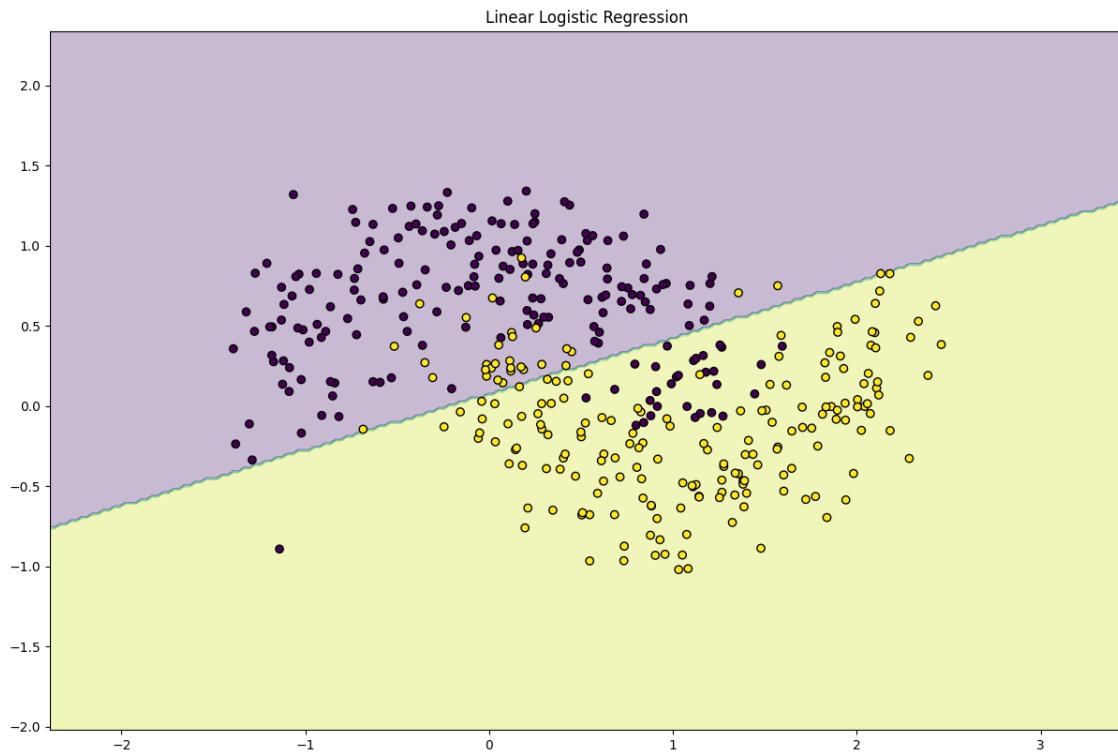
```
[15]: kernel_model = KernelLogisticRegression(gamma=5, lr=0.01, epochs=500)
kernel_model.fit(X, y)
```

```
[15]: KernelLogisticRegression(epochs=500, gamma=5, lr=0.01)
```

```
[16]: DecisionBoundaryDisplay.from_estimator(
    linear_model,
    X,
    response_method="predict",
    alpha=0.3,
    grid_resolution=200)
```

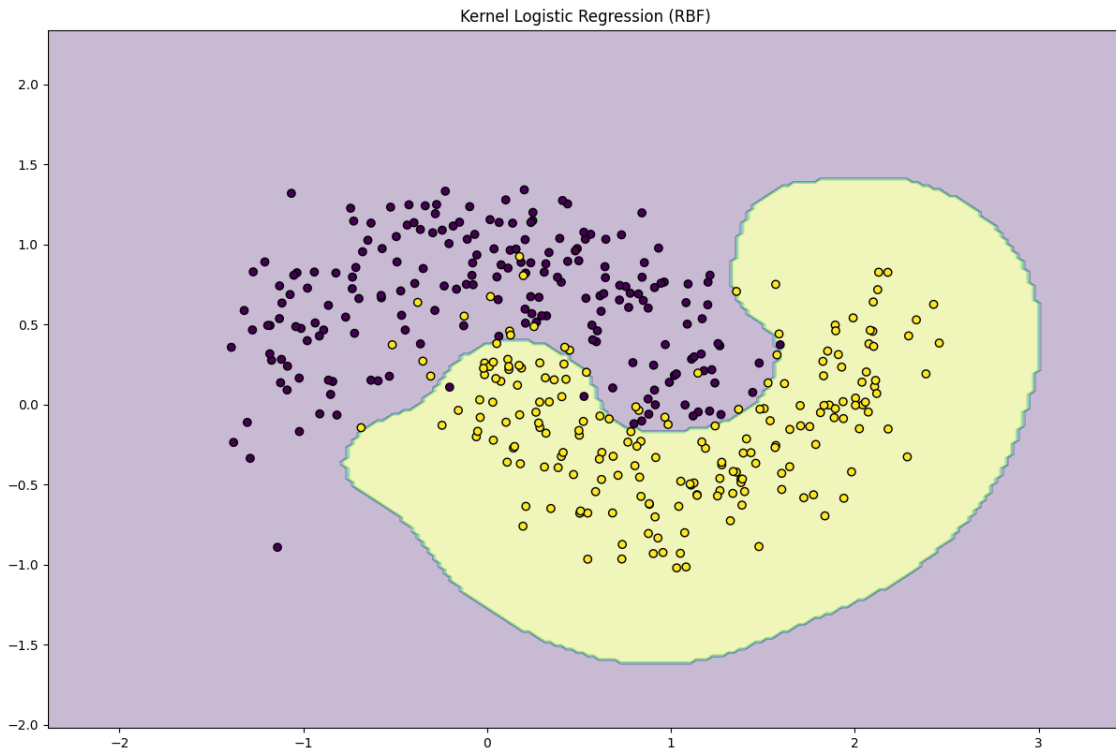
```
)

plt.scatter(X[:, 0], X[:, 1], c=y, edgecolor="k")
plt.title("Linear Logistic Regression")
plt.show()
```



```
[17]: DecisionBoundaryDisplay.from_estimator(
    kernel_model,
    X,
    response_method="predict",
    alpha=0.3,
    grid_resolution=200
)

plt.scatter(X[:, 0], X[:, 1], c=y, edgecolor="k")
plt.title("Kernel Logistic Regression (RBF)")
plt.show()
```



1.4.4 VC_dimension

```
[9]: d.y = np.where(d.y == 1, 1, 0)
X_train, y_train = d.X_train, d.y_train
X_val, y_val      = d.X_val, d.y_val
X_test, y_test    = d.X_test, d.y_test
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val    = scaler.transform(X_val)
X_test   = scaler.transform(X_test)

# ===== VC dimension =====
D = d.n_features
vc_dim = D + 1

print("===== VC DIMENSION (Covtype Dataset) =====")
print(f"Number of features (D): {D}")
print(f"Estimated VC dimension: {vc_dim}")

# ===== SRM =====
C_values = np.logspace(-3, 2, 6) # đẹp hơn list thường

train_acc, val_acc, test_acc = [], [], []
```

```

print("\n===== SRM EXPERIMENT =====")

for C in C_values:
    model = LogisticRegression(C=C, max_iter=500)

    model.fit(X_train, y_train)

    train = accuracy_score(y_train, model.predict(X_train))
    val    = accuracy_score(y_val, model.predict(X_val))
    test   = accuracy_score(y_test, model.predict(X_test))

    train_acc.append(train)
    val_acc.append(val)
    test_acc.append(test)

    print(f"C = {C:>7.4f} | Train = {train:.4f} | Val = {val:.4f} | Test = {test:.4f}")

# ===== Select best model =====
best_idx = np.argmax(val_acc)
best_C = C_values[best_idx]

print("\n===== SRM RESULT =====")
print(f"Best C (from validation): {best_C}")
print(f"Best Validation Accuracy: {val_acc[best_idx]:.4f}")
print(f"Test Accuracy (best model): {test_acc[best_idx]:.4f}")

# ===== Plot =====
plt.figure(figsize=(8, 5))

plt.plot(C_values, train_acc, marker='o', label="Train")
plt.plot(C_values, val_acc, marker='o', label="Validation")
plt.plot(C_values, test_acc, marker='o', label="Test")

plt.xscale('log')
plt.xlabel("Regularization Strength (C)")
plt.ylabel("Accuracy")
plt.title("Structural Risk Minimization on Covtype Dataset")

plt.legend()
plt.grid(alpha=0.3)
plt.show()

```

```

===== VC DIMENSION (Covtype Dataset) =====
Number of features (D): 54
Estimated VC dimension: 55

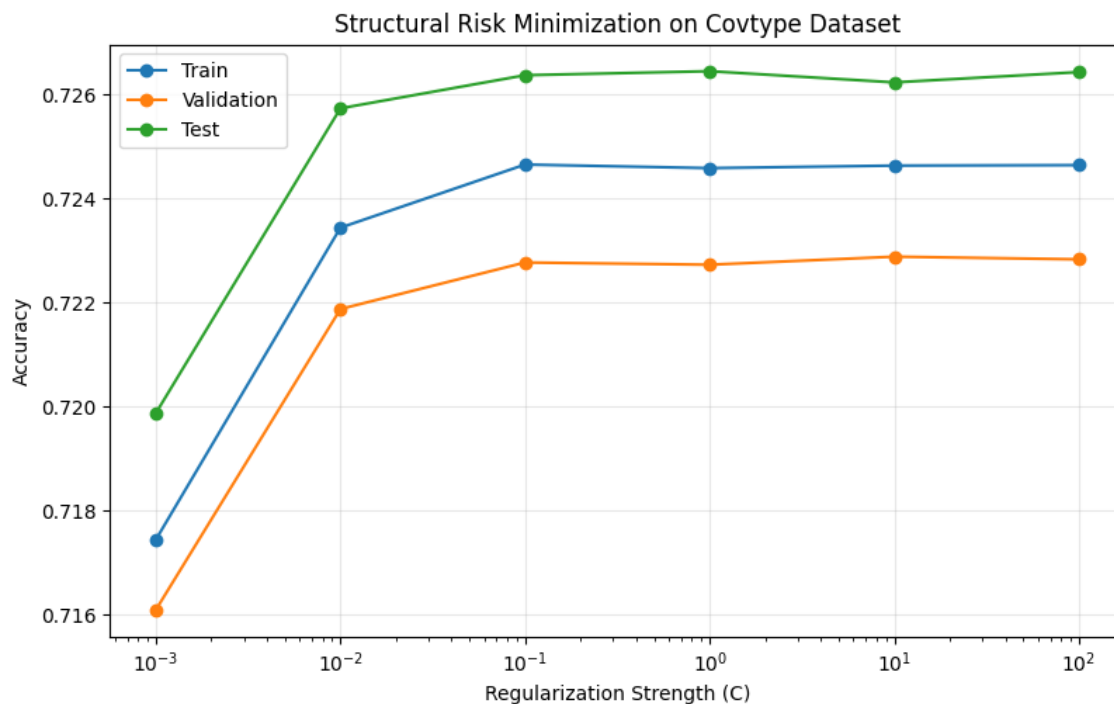
```

===== SRM EXPERIMENT =====

```
C = 0.0010 | Train = 0.7174 | Val = 0.7161 | Test = 0.7199
C = 0.0100 | Train = 0.7234 | Val = 0.7219 | Test = 0.7257
C = 0.1000 | Train = 0.7247 | Val = 0.7228 | Test = 0.7264
C = 1.0000 | Train = 0.7246 | Val = 0.7227 | Test = 0.7265
C = 10.0000 | Train = 0.7246 | Val = 0.7229 | Test = 0.7262
C = 100.0000 | Train = 0.7246 | Val = 0.7228 | Test = 0.7264
```

===== SRM RESULT =====

```
Best C (from validation): 10.0
Best Validation Accuracy: 0.7229
Test Accuracy (best model): 0.7262
```



1.5 Evaluation

```
[39]: logger = get_logger("logger", "log.txt")
      # logger = None

[40]: models = {
      "Softmax": get_pipeline(SoftmaxClassifier(store_history=False)),
      "LDA": get_pipeline(LDA()),
      "QDA": get_pipeline(QDA()),
      "Weighted Softmax": get_pipeline(WeightedSoftmaxClassifier(store_history=False))
    }
```

1.5.1 Evaluate on test set, cross-validation, McNemar test

```
[41]: evaluator = Evaluator(logger=logger,log_model=True)
```

On test set

```
[42]: # ===== TEST =====

test_result = evaluator.compare_models_test(
    models,
    d.X_train,
    d.y_train,
    d.X_test,
    d.y_test
)

# test_result.round(4).to_csv("classification-compare-models-test.csv",
    ↪index=False)
display(test_result)
```

	Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)	\
0	Softmax	0.714801	0.590134	0.430559	0.431943	
1	LDA	0.681514	0.485564	0.576581	0.509563	
2	QDA	0.242137	0.358942	0.554776	0.256033	
3	Weighted Softmax	0.592007	0.460329	0.697000	0.491955	

	ROC AUC	Precision_per_class	\
0	None	[0.7115056748989235, 0.7407516532442873, 0.609...	
1	None	[0.7053935234827364, 0.7585891941263262, 0.625...	
2	None	[0.6646894161556918, 0.8815573770491804, 0.400...	
3	None	[0.6809280697969847, 0.7972182971767379, 0.662...	

	Recall_per_class	\
0	[0.6895062311178247, 0.7986939639957642, 0.879...	
1	[0.6210819486404834, 0.7622308506883163, 0.531...	
2	[0.235389916918429, 0.11388986939639957, 0.766...	
3	[0.6705296450151057, 0.5078362160254147, 0.533...	

	F1_per_class	\
0	[0.7003332294488529, 0.768632379917114, 0.7196...	
1	[0.6605582889848378, 0.7604056623705895, 0.574...	
2	[0.3476608798717144, 0.20171928727727415, 0.52...	
3	[0.6756888534766735, 0.6204435436050586, 0.590...	

	Confusion Matrix
0	[[29213, 12181, 43, 0, 2, 0, 929], [9793, 4525...
1	[[26314, 11540, 43, 0, 48, 32, 4391], [10227, ...
2	[[9973, 840, 378, 0, 11997, 400, 18780], [4868...
3	[[28409, 6906, 38, 1, 1964, 292, 4758], [12813...

Cross-validation

```
[43]: # ===== CV =====

cv_result = evaluator.compare_models_cv(
    models,
    d.X_train,
    d.y_train
)

# cv_result.round(4).to_csv("classification-compare-models-cv.csv", index=False)
display(cv_result)
```

	Model	Accuracy	Precision (macro)	Recall (macro)	\
0	Softmax	0.7122 ± 0.0009	0.5703 ± 0.0293	0.4212 ± 0.0015	
1	LDA	0.6795 ± 0.0013	0.4877 ± 0.0046	0.5791 ± 0.0060	
2	QDA	0.2427 ± 0.0009	0.3589 ± 0.0043	0.5548 ± 0.0030	
3	Weighted Softmax	0.5903 ± 0.0011	0.4556 ± 0.0015	0.6899 ± 0.0040	
	F1 (macro)				
0		0.4155 ± 0.0025			
1		0.5108 ± 0.0054			
2		0.2572 ± 0.0018			
3		0.4885 ± 0.0015			

McNemar test

```
[44]: # ===== STATISTICAL TEST =====

# IMPORTANT: fit trước trên train
for model in models.values():
    model.fit(d.X_train, d.y_train)

stat_result = evaluator.compare_models_statistical(
    models,
    d.X_test,
    d.y_test
)

# stat_result.round(4).to_csv("classification-mcnemar-test.csv", index=False)

display(stat_result)
```

	Model A	Model B	McNemar statistic	p-value
0	Softmax	LDA	4700.0	1.126516e-250
1	Softmax	QDA	5573.0	0.000000e+00
2	Softmax	Weighted Softmax	8907.0	0.000000e+00
3	LDA	QDA	5508.0	0.000000e+00
4	LDA	Weighted Softmax	6211.0	0.000000e+00
5	QDA	Weighted Softmax	3379.0	0.000000e+00

1.5.2 Visualize

```
[45]: def plot_all_evaluation(visualizer, models, class_names, X_test, y_test):

    # =====
    # BUILD models_results (NO FIT, ONLY PREDICT)
    # =====
    models_results = {}

    for name, model in models.items():

        y_pred = model.predict(X_test)
        y_score = model.predict_proba(X_test)

        models_results[name] = {
            "model": model,
            "y_pred": y_pred,
            "y_score": y_score
        }

    # =====
    model_names = list(models_results.keys())
    n_models = len(model_names)

    n_cols = 2
    n_rows = int(np.ceil(n_models / n_cols))

    # =====
    # helper: handle axes safely
    # =====
    def safe_axes():
        fig, axes = plt.subplots(n_rows, n_cols)
        axes = np.array(axes).reshape(-1)
        return fig, axes

    # =====
    # 1. CONFUSION MATRIX
    # =====
    fig, axes = safe_axes()

    for i, name in enumerate(model_names):
        visualizer.plot_confusion_matrix(
            y_test,
            models_results[name]["y_pred"],
            class_names,
            ax=axes[i]
        )
        axes[i].set_title(f"{name} - Confusion Matrix")
```

```

for j in range(n_models, len(axes)):
    axes[j].set_visible(False)

fig.suptitle("Confusion Matrix Comparison")
plt.show()

# =====
# 2. ROC CURVE
# =====
fig, axes = safe_axes()

for i, name in enumerate(model_names):
    visualizer.plot_roc_curve(
        y_test,
        models_results[name]["y_score"],
        class_names,
        ax=axes[i]
    )
    axes[i].set_title(f"{name} - ROC")

for j in range(n_models, len(axes)):
    axes[j].set_visible(False)

fig.suptitle("ROC Curve Comparison")
plt.show()

# =====
# 3. PR CURVE
# =====
fig, axes = safe_axes()

for i, name in enumerate(model_names):
    visualizer.plot_precision_recall_curve(
        y_test,
        models_results[name]["y_score"],
        class_names,
        ax=axes[i]
    )
    axes[i].set_title(f"{name} - PR Curve")

for j in range(n_models, len(axes)):
    axes[j].set_visible(False)

fig.suptitle("Precision-Recall Curve Comparison")
plt.show()

```

```

# =====
# 4. CALIBRATION
# =====

fig, axes = safe_axes()

for i, name in enumerate(model_names):
    visualizer.plot_calibration_curve(
        y_test,
        models_results[name]["y_score"],
        class_names,
        ax=axes[i]
    )
    axes[i].set_title(f"{name} - Calibration")

for j in range(n_models, len(axes)):
    axes[j].set_visible(False)

fig.suptitle("Calibration Curve Comparison")
plt.show()

# =====
# 5. LOSS CURVE (PIPELINE SAFE + NO EMPTY CRASH)
# =====

fig, axes = safe_axes()

for i, name in enumerate(model_names):

    model = models_results[name]["model"]

    predictor = model.steps[-1][1] if hasattr(model, "steps") else model

    if hasattr(predictor, "train_loss_history_") and len(predictor.
↪train_loss_history_) > 0:

        val_loss = getattr(predictor, "val_loss_history_", None)
        if val_loss is not None and len(val_loss) == 0:
            val_loss = None

        visualizer.plot_loss_epochs(
            predictor.train_loss_history_,
            val_loss,
            ax=axes[i]
        )
        axes[i].set_title(f"{name} - Loss Curve")

    else:
        axes[i].text(0.5, 0.5, "No GD history",

```

```

        ha="center", va="center")
    axes[i].set_title(f"{name} - Loss Curve")

    for j in range(n_models, len(axes)):
        axes[j].set_visible(False)

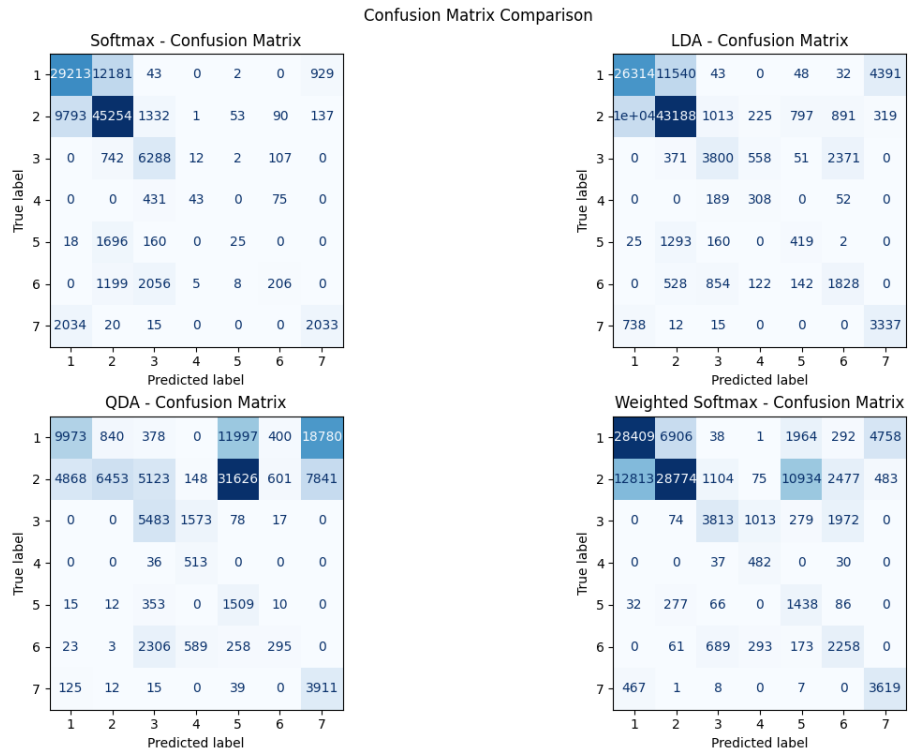
    fig.suptitle("Training Loss Comparison")
    plt.show()

```

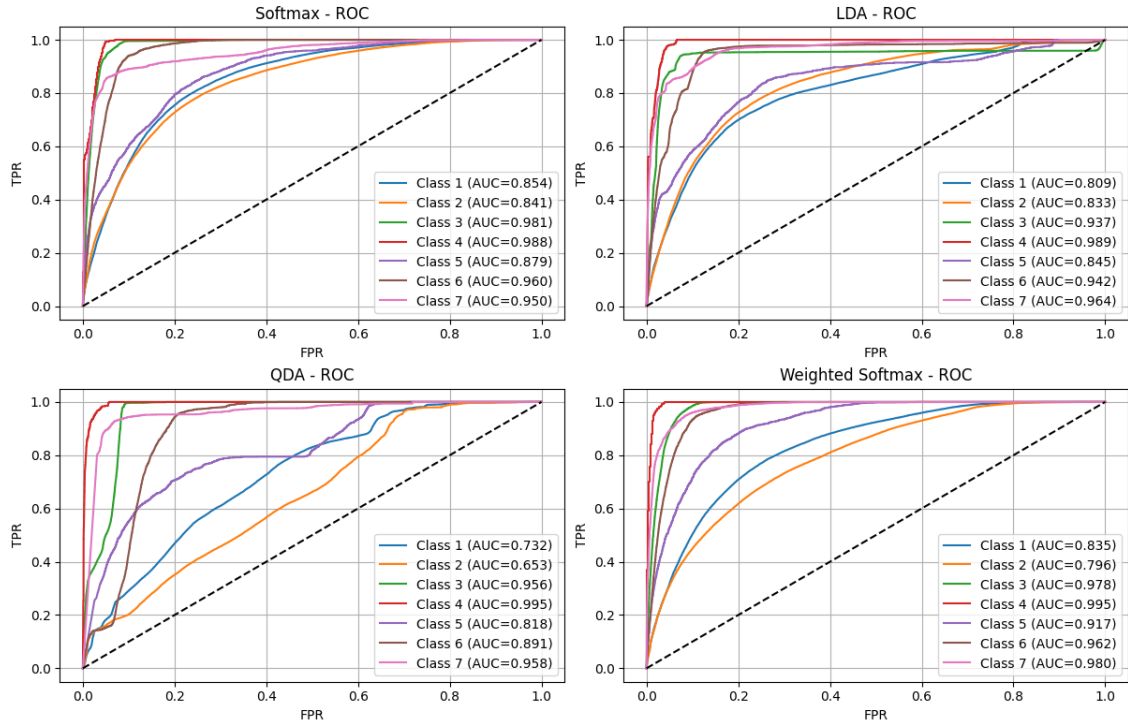
```

[46]: visualizer = Visualizer()
      plot_all_evaluation(visualizer, models, d.classes, d.X_test, d.y_test)

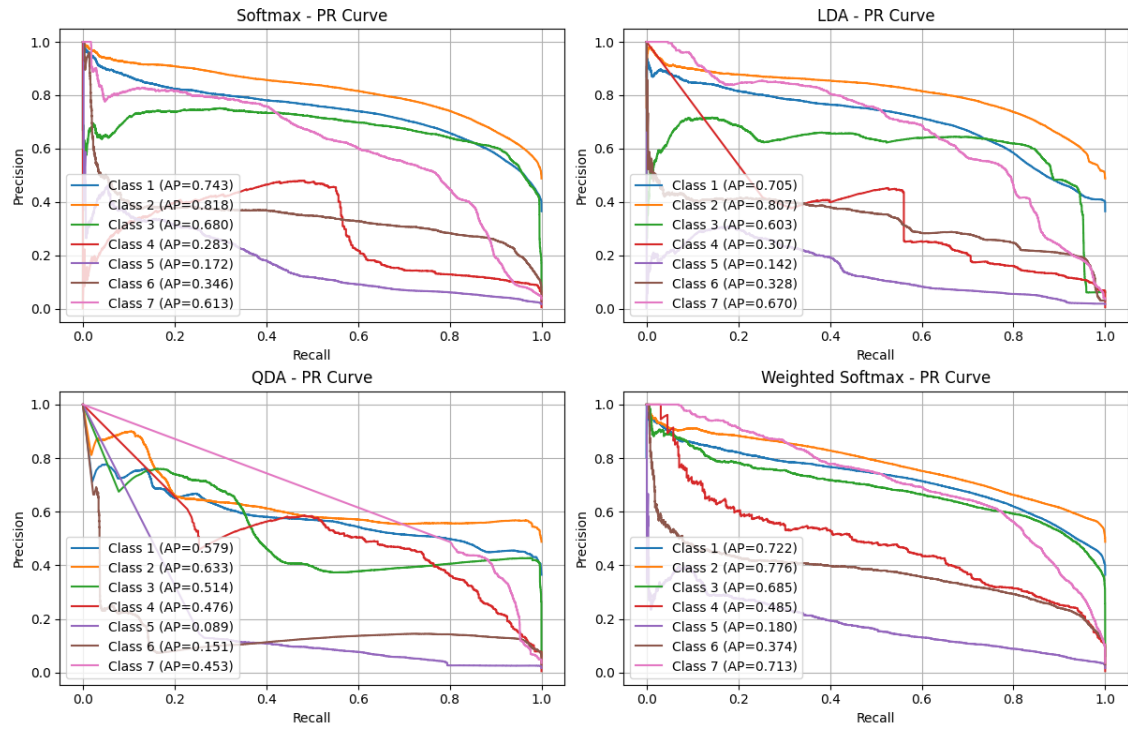
```



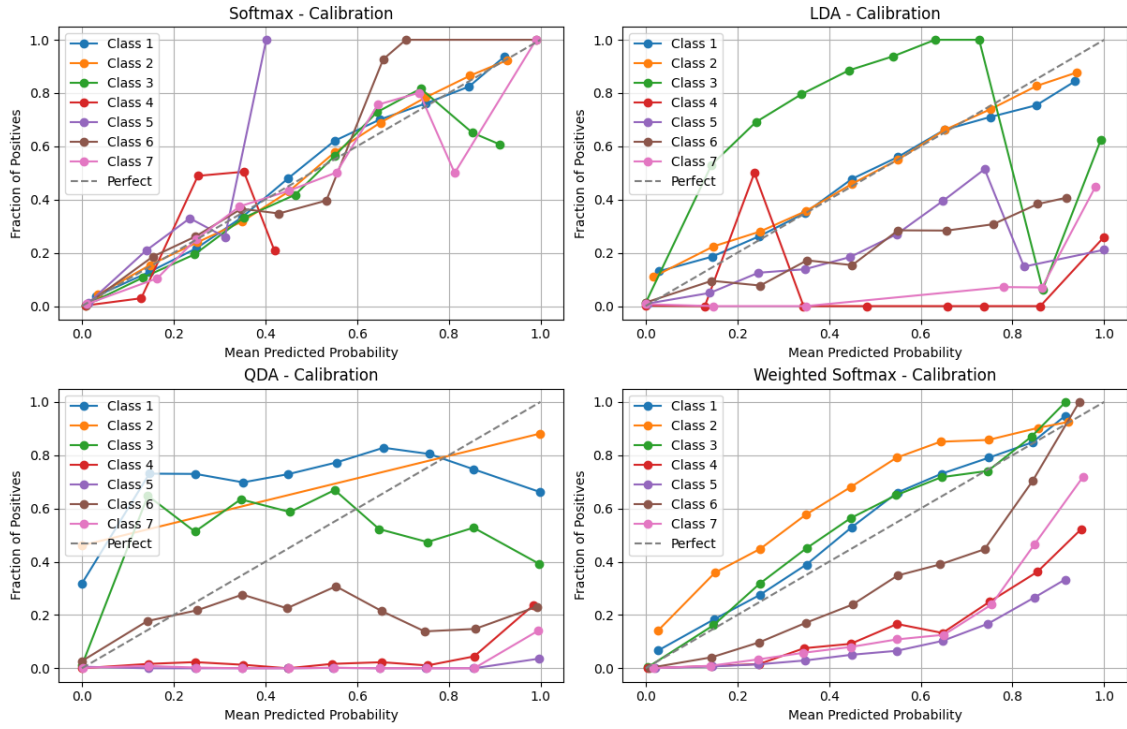
ROC Curve Comparison



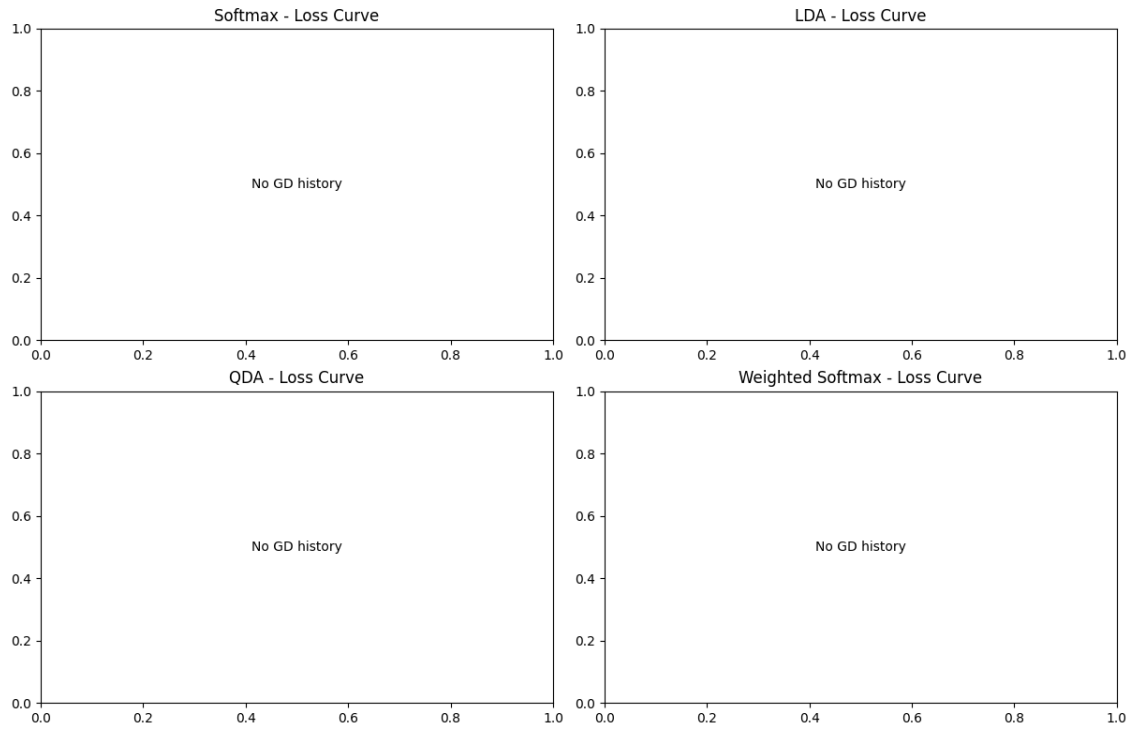
Precision-Recall Curve Comparison



Calibration Curve Comparison



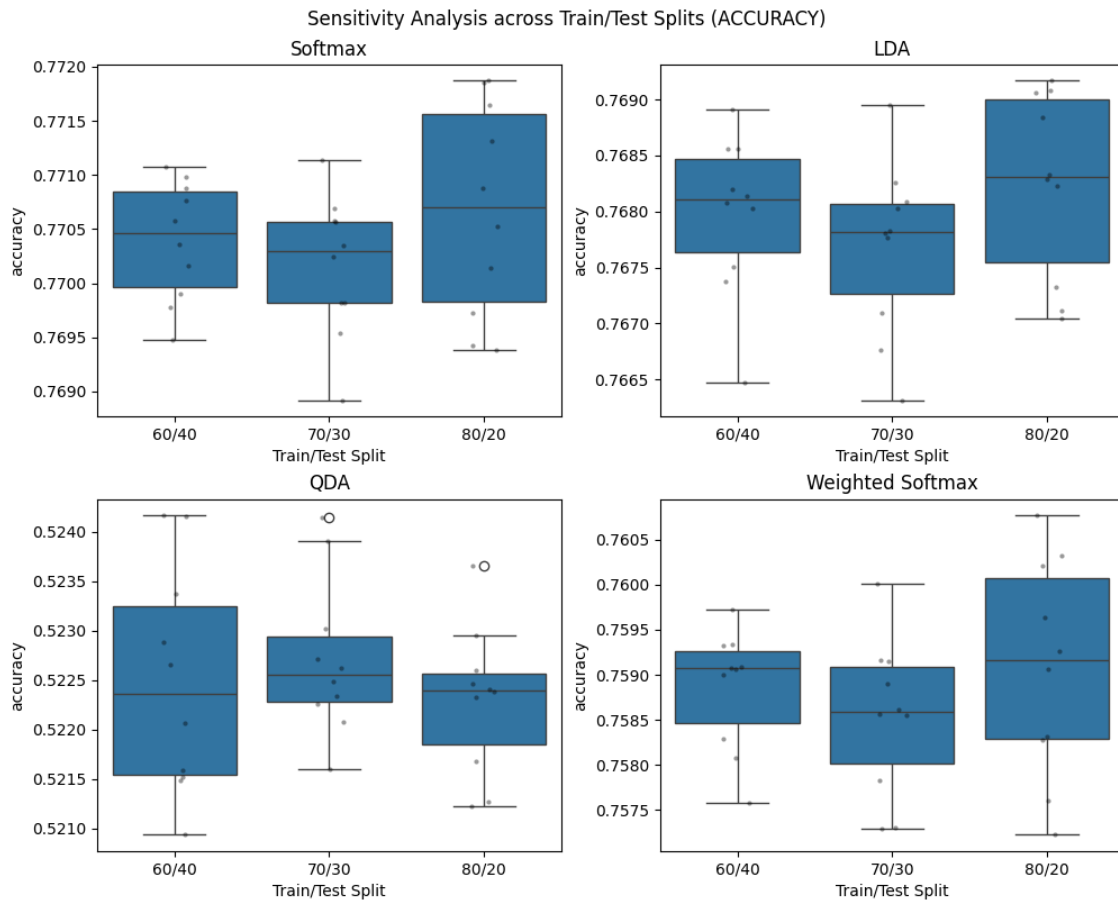
Training Loss Comparison



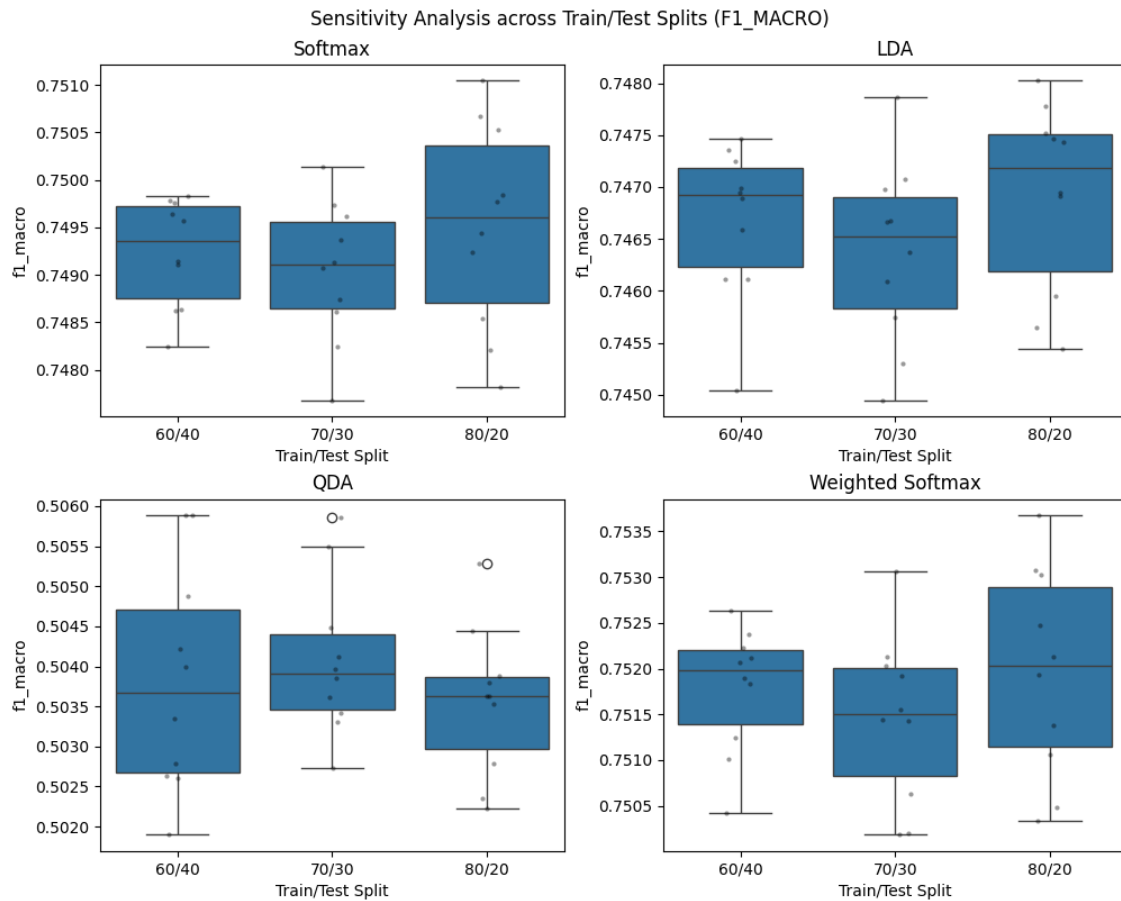
Split sensitivity

	model	split	seed	accuracy	precision_macro	\
0	Softmax	60/40	383329927	0.771076	0.753536	
1	LDA	60/40	383329927	0.768912	0.751115	
2	QDA	60/40	383329927	0.524154	0.697860	
3	Weighted Softmax	60/40	383329927	0.758998	0.750848	
4	Softmax	60/40	3324115916	0.770573	0.752869	
..	...	...	...	...	...	
115	Weighted Softmax	80/20	2342600226	0.757227	0.749857	
116	Softmax	80/20	1904449481	0.771873	0.754288	
117	LDA	80/20	1904449481	0.769171	0.751322	
118	QDA	80/20	1904449481	0.522947	0.697091	
119	Weighted Softmax	80/20	1904449481	0.758319	0.750761	
	recall_macro	f1_macro				
0	0.746799	0.749757				
1	0.744596	0.747464				
2	0.621817	0.505886				
3	0.769408	0.751834				
4	0.746916	0.749567				
..	...	...				
115	0.768612	0.750340				
116	0.748427	0.751044				
117	0.745380	0.748024				
118	0.620800	0.504440				
119	0.769523	0.751385				

```
/home/quihoang/Projects/supervised-learning-
project/code/Part2_Classification/sensitivity_robustness.py:616: RuntimeWarning:
More than 20 figures have been opened. Figures created through the pyplot
interface (`matplotlib.pyplot.figure`) are retained until explicitly closed and
may consume too much memory. (To control this warning, see the rcParam
`figure.max_open_warning`). Consider using `matplotlib.pyplot.close()`.
    fig, axes = plt.subplots(
```



```
[49]: fig = plot_split_boxplots(df, metric="f1_macro")
plt.show()
```



Gaussian noise sensitivity

```
[50]: result = noise_injection_analysis(models, copy.deepcopy(d),
    ↪task="classification")
# result.round(4).to_csv("classification-sensitivity-gaussian-noise.csv",
    ↪index=False)
display(result)
```

	model	sigma	accuracy	precision_macro	recall_macro	\
0	Softmax	0.0	0.769825	0.752193	0.745191	
1	LDA	0.0	0.766942	0.748946	0.742379	
2	QDA	0.0	0.521389	0.697164	0.619710	
3	Weighted Softmax	0.0	0.759120	0.750890	0.769423	
4	Softmax	0.1	0.763371	0.745120	0.737622	
5	LDA	0.1	0.759671	0.741035	0.733393	
6	QDA	0.1	0.678304	0.711707	0.720198	
7	Weighted Softmax	0.1	0.756917	0.748683	0.767056	
8	Softmax	0.5	0.758010	0.739675	0.728781	
9	LDA	0.5	0.753440	0.734546	0.723652	
10	QDA	0.5	0.734120	0.725360	0.741870	

11	Weighted Softmax	0.5	0.755841	0.746559	0.764354
12	Softmax	1.0	0.757992	0.739797	0.728124
13	LDA	1.0	0.754344	0.735710	0.723940
14	QDA	1.0	0.734774	0.724189	0.739870
15	Weighted Softmax	1.0	0.755523	0.746386	0.764234

	f1_macro	is_clean	accuracy_sens_abs	accuracy_sens_rel	\
0	0.748247	True	NaN	NaN	
1	0.745261	True	NaN	NaN	
2	0.502416	True	NaN	NaN	
3	0.751925	True	NaN	NaN	
4	0.740844	False	0.006454	0.008384	
5	0.736654	False	0.007272	0.009481	
6	0.677781	False	-0.156915	-0.300956	
7	0.749659	False	0.002203	0.002902	
8	0.733139	False	0.011816	0.015348	
9	0.727975	False	0.013502	0.017605	
10	0.725988	False	-0.212731	-0.408008	
11	0.748078	False	0.003279	0.004319	
12	0.732725	False	0.011833	0.015371	
13	0.728539	False	0.012599	0.016427	
14	0.725736	False	-0.213385	-0.409263	
15	0.747819	False	0.003597	0.004739	

	precision_macro_sens_abs	precision_macro_sens_rel	recall_macro_sens_abs	\
0	NaN	NaN	NaN	
1	NaN	NaN	NaN	
2	NaN	NaN	NaN	
3	NaN	NaN	NaN	
4	0.007073	0.009403	0.007568	
5	0.007911	0.010563	0.008986	
6	-0.014543	-0.020860	-0.100488	
7	0.002207	0.002939	0.002367	
8	0.012518	0.016642	0.016409	
9	0.014399	0.019226	0.018728	
10	-0.028196	-0.040444	-0.122160	
11	0.004331	0.005768	0.005070	
12	0.012396	0.016480	0.017067	
13	0.013236	0.017672	0.018439	
14	-0.027026	-0.038765	-0.120160	
15	0.004503	0.005997	0.005189	

	recall_macro_sens_rel	f1_macro_sens_abs	f1_macro_sens_rel
0	NaN	NaN	NaN
1	NaN	NaN	NaN
2	NaN	NaN	NaN
3	NaN	NaN	NaN
4	0.010156	0.007403	0.009894

5	0.012105	0.008607	0.011549
6	-0.162154	-0.175364	-0.349042
7	0.003077	0.002266	0.003013
8	0.022020	0.015109	0.020192
9	0.025226	0.017286	0.023195
10	-0.197124	-0.223572	-0.444993
11	0.006589	0.003847	0.005116
12	0.022903	0.015523	0.020745
13	0.024838	0.016722	0.022437
14	-0.193897	-0.223320	-0.444492
15	0.006745	0.004106	0.005460

Feature corruption with imputers sensitivity

```
[52]: result = feature_corruption_analysis(models, copy.deepcopy(d), imputers=[
    ↪ ["mean", "median"], task="classification")
# result.round(4).to_csv("classification-sensitivity-imputer.csv", index=False)
display(result)
```

	model	imputer	missing_rate	is_clean	accuracy \
0	Softmax	none	0.0	True	0.769825
1	LDA	none	0.0	True	0.766942
2	QDA	none	0.0	True	0.521389
3	Weighted Softmax	none	0.0	True	0.759120
4	Softmax	mean	0.1	False	0.755686
5	LDA	mean	0.1	False	0.746814
6	QDA	mean	0.1	False	0.527844
7	Weighted Softmax	mean	0.1	False	0.742167
8	Softmax	median	0.1	False	0.755979
9	LDA	median	0.1	False	0.745488
10	QDA	median	0.1	False	0.514393
11	Weighted Softmax	median	0.1	False	0.742270
12	Softmax	mean	0.2	False	0.743621
13	LDA	mean	0.2	False	0.736298
14	QDA	mean	0.2	False	0.549943
15	Weighted Softmax	mean	0.2	False	0.728174
16	Softmax	median	0.2	False	0.744447
17	LDA	median	0.2	False	0.734172
18	QDA	median	0.2	False	0.549478
19	Weighted Softmax	median	0.2	False	0.727847
20	Softmax	mean	0.3	False	0.732752
21	LDA	mean	0.3	False	0.724878
22	QDA	mean	0.3	False	0.531991
23	Weighted Softmax	mean	0.3	False	0.711393
24	Softmax	median	0.3	False	0.733174
25	LDA	median	0.3	False	0.722666
26	QDA	median	0.3	False	0.531991
27	Weighted Softmax	median	0.3	False	0.711359

	precision_macro	recall_macro	f1_macro	accuracy_sens_abs \
0	0.752193	0.745191	0.748247	NaN
1	0.748946	0.742379	0.745261	NaN
2	0.697164	0.619710	0.502416	NaN
3	0.750890	0.769423	0.751925	NaN
4	0.737069	0.726168	0.730511	0.014139
5	0.727426	0.714383	0.719274	0.020129
6	0.697882	0.624432	0.510622	-0.006454
7	0.734977	0.752683	0.734952	0.016953
8	0.737339	0.726791	0.731023	0.013846
9	0.725917	0.712913	0.717778	0.021454
10	0.696827	0.614652	0.493350	0.006996
11	0.735731	0.753664	0.735323	0.016850
12	0.724597	0.707400	0.713276	0.026204
13	0.717557	0.694425	0.701212	0.030645
14	0.695213	0.639071	0.538705	-0.028553
15	0.719615	0.735782	0.719971	0.030946
16	0.725620	0.708090	0.714057	0.025378
17	0.715432	0.690941	0.697843	0.032770
18	0.695400	0.638806	0.538109	-0.028089
19	0.720948	0.737683	0.720378	0.031273
20	0.713863	0.688994	0.695888	0.037073
21	0.707235	0.673478	0.680640	0.042064
22	0.691273	0.625820	0.516850	-0.010602
23	0.701852	0.716431	0.702232	0.047727
24	0.714482	0.689185	0.696150	0.036651
25	0.705557	0.668957	0.676007	0.044276
26	0.691273	0.625820	0.516850	-0.010602
27	0.704992	0.720639	0.703690	0.047761

	accuracy_sens_rel	precision_macro_sens_abs	precision_macro_sens_rel \
0	NaN	NaN	NaN
1	NaN	NaN	NaN
2	NaN	NaN	NaN
3	NaN	NaN	NaN
4	0.018367	0.015123	0.020106
5	0.026245	0.021520	0.028734
6	-0.012379	-0.000718	-0.001030
7	0.022333	0.015913	0.021192
8	0.017986	0.014854	0.019747
9	0.027973	0.023029	0.030749
10	0.013419	0.000337	0.000483
11	0.022197	0.015159	0.020187
12	0.034039	0.027596	0.036688
13	0.039957	0.031389	0.041911
14	-0.054764	0.001951	0.002798
15	0.040765	0.031274	0.041650
16	0.032966	0.026572	0.035327

17	0.042728	0.033513	0.044747
18	-0.053873	0.001764	0.002530
19	0.041196	0.029941	0.039874
20	0.048158	0.038330	0.050958
21	0.054847	0.041711	0.055693
22	-0.020334	0.005891	0.008450
23	0.062871	0.049037	0.065306
24	0.047610	0.037710	0.050134
25	0.057731	0.043388	0.057933
26	-0.020334	0.005891	0.008450
27	0.062917	0.045898	0.061125

	recall_macro_sens_abs	recall_macro_sens_rel	f1_macro_sens_abs \
0	NaN	NaN	NaN
1	NaN	NaN	NaN
2	NaN	NaN	NaN
3	NaN	NaN	NaN
4	0.019022	0.025527	0.017737
5	0.027996	0.037711	0.025987
6	-0.004722	-0.007619	-0.008205
7	0.016741	0.021757	0.016973
8	0.018400	0.024692	0.017224
9	0.029466	0.039691	0.027483
10	0.005058	0.008162	0.009066
11	0.015759	0.020482	0.016602
12	0.037791	0.050713	0.034971
13	0.047954	0.064596	0.044049
14	-0.019361	-0.031242	-0.036289
15	0.033641	0.043722	0.031953
16	0.037101	0.049787	0.034190
17	0.051438	0.069288	0.047418
18	-0.019096	-0.030814	-0.035693
19	0.031741	0.041253	0.031547
20	0.056197	0.075412	0.052360
21	0.068901	0.092811	0.064621
22	-0.006110	-0.009859	-0.014434
23	0.052992	0.068873	0.049693
24	0.056006	0.075156	0.052098
25	0.073423	0.098902	0.069254
26	-0.006110	-0.009859	-0.014434
27	0.048785	0.063404	0.048235

	f1_macro_sens_rel
0	NaN
1	NaN
2	NaN
3	NaN
4	0.023704

```

5         0.034869
6        -0.016332
7         0.022572
8         0.023019
9         0.036877
10        0.018046
11        0.022079
12        0.046737
13        0.059106
14       -0.072229
15        0.042495
16        0.045694
17        0.063626
18       -0.071042
19        0.041955
20        0.069976
21        0.086710
22       -0.028729
23        0.066087
24        0.069626
25        0.092926
26       -0.028729
27        0.064148

```

Loss curves

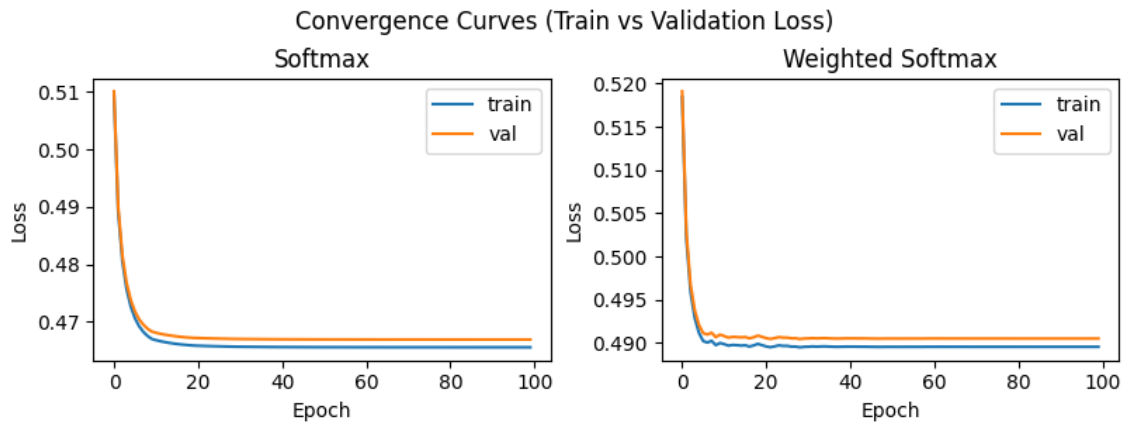
```

[53]: gd_models = {
        name: model
        for name, model in models.items()
        if isinstance(model.named_steps["predictor"], BaseGDModel)
    }
    display(gd_models)

{'Softmax': Pipeline(steps=[('features', None), ('scaler', StandardScaler()),
                             ('predictor', SoftmaxClassifier(store_history=False))]),
 'Weighted Softmax': Pipeline(steps=[('features', None), ('scaler',
↳StandardScaler()),
                                       ('predictor', WeightedSoftmaxClassifier(store_history=False))])}

[54]: epochs = [100, 100]
    fig = plot_loss_curves(gd_models, copy.deepcopy(d), epochs)
    plt.show()

```



1.5.4 Benchmarks

```
[55]: bm_models = {
        name: model
        for name, model in models.items()
        if isinstance(model.named_steps["predictor"], BaseModel)
    }
    display(bm_models)

{'Softmax': Pipeline(steps=[('features', None), ('scaler', StandardScaler()),
                             ('predictor', SoftmaxClassifier(store_history=False))]),
 'LDA': Pipeline(steps=[('features', None), ('scaler', StandardScaler()),
                          ('predictor', LDA())]),
 'QDA': Pipeline(steps=[('features', None), ('scaler', StandardScaler()),
                          ('predictor', QDA())]),
 'Weighted Softmax': Pipeline(steps=[('features', None), ('scaler',
                                                           StandardScaler()),
                                     ('predictor', WeightedSoftmaxClassifier(store_history=False))])}

[56]: result = benchmark_models(bm_models, d)
      # result.round(4).to_csv("classification-benchmark-models.csv")
      display(result)
```

	model	train_time_mean	train_time_std	train_memory_mean \
0	Softmax	29.405160	0.110729	442574877.8
1	LDA	2.437492	0.023287	876553292.6
2	QDA	2.162390	0.088970	382832913.2
3	Weighted Softmax	28.709978	0.115723	442574417.4

	train_memory_std	inference_time_mean	inference_time_std \
0	886.912713	0.028295	0.000334
1	233.023261	0.007771	0.001994
2	18.400000	0.102984	0.009362

3	409.177028	0.031238	0.005633
---	------------	----------	----------

	inference_memory_mean	inference_memory_std
0	57704104.0	0.000000
1	2789844.0	0.000000
2	152459992.4	22.535306
3	57704104.0	0.000000

[]: